



raffle.fun

WHITEPAPER · VERSION 1.0

raffle.fun

A Whitepaper for Onchain NFT Raffles

How prize custody, ticket ownership, random selection, payouts, refunds, and recovery work.

Networks Ethereum & Base (undeployed)

Commit a2120f5e

Date August 9, 2026

Status Unaudited

Heesho · raffle.fun contributors

ABOUT THIS DOCUMENT

Document and review record

FIELD	VALUE
Title	raffle.fun: A Whitepaper for Onchain NFT Raffles
Author	Heesho
Document version	1.0
Protocol version reviewed	raffle.fun v1 (monorepo 0.1.0)
Reviewed Git commit	<code>a2120f5e163dc3641d9864773febbfedca047edb</code>
Publication date	August 9, 2026
Target networks	Ethereum mainnet and Base (an Ethereum layer-2 network), plus their test networks
Deployment status	Undeployed. No testnet or mainnet deployment is recorded at the reviewed commit.
Security review status	Internal testing, fuzzing, invariant testing, static analysis, and review only. No independent audit has been performed.
Compiler / dependencies	Solidity 0.8.36 · OpenZeppelin 5.6.1 · Pyth Entropy SDK 2.2.1
License	MIT (repository), CC BY 4.0 (this document text)

DISCLAIMER

This whitepaper explains how the raffle.fun smart contracts work at one reviewed source-code commit. It is an explanatory document, not an offer, a promise of future performance, or an inducement to participate. The protocol distributes a prize by chance; gambling, sweepstakes, lottery, consumer-protection, sanctions, tax, and licensing laws treat chance-based prize systems differently across jurisdictions, and nothing here is legal, tax, investment, or financial advice. Anyone who operates or uses the protocol must obtain advice for their own jurisdiction. Smart-contract testing and internal review reduce risk but cannot eliminate it, and this document describes unaudited software: do not commit assets you cannot afford to lose unless and until the protocol has received appropriate independent review and production validation. Raffle tickets are entries in a prize drawing. They are not investments, they carry no expectation of profit, and most tickets win nothing.

CONTENTS

What is in this whitepaper

How to Read This Whitepaper	4
PART I: THE IDEA	
1 Executive Summary	6
2 raffle.fun in 60 Seconds	7
3 The Problem	8
4 The Core Idea	10
5 What "Onchain" Means	10
PART II: THE PEOPLE AND THE ASSETS	
6 Participants	13
7 The Prize NFT	14
8 The Payment Token	16
9 Ticket NFTs	16
PART III: HOW A RAFFLE WORKS	
10 Creating a Raffle	19
11 Buying Tickets	20
12 The Raffle Timeline	21
13 The Minimum-Ticket Threshold	22
14 Requesting the Draw	23
15 How Randomness Works	24
16 Freezing the Winner	26
PART IV: OUTCOMES AND MONEY	
17 Outcome Overview	29
18 Threshold Met: the NFT Branch	30
19 Threshold Missed: the Cash Branch	31
20 No Sales and Early Cancellation	31
21 Failed Draws, Timeouts, and Refunds	32
22 Fees and Accounting	34
23 Why Claims Are Pulled	35
PART V: ARCHITECTURE	
24 The Contract System	38
25 Architecture Diagram	39
26 What Is Authoritative?	40
27 Deployment and Versioning	40
PART VI: SECURITY AND TRUST	
28 Security Goals	43
29 Security Design Choices	44
30 Administrator Powers	45
31 The Trust and Dependency Map	46
32 Threat Model	48
33 What the Protocol Cannot Guarantee	49
34 Testing and Review	50
PART VII: USER GUIDES	
35 Sponsor Walkthrough	52
36 Ticket Buyer Walkthrough	53
37 Winner Walkthrough	54
38 Safety Checklists	55
PART VIII: FREQUENTLY ASKED QUESTIONS	
Frequently Asked Questions	56
PART IX: CONCLUSION	
39 What raffle.fun Makes Possible	61
40 Final Takeaways	61
APPENDICES: TECHNICAL REFERENCE	
A Exact State Machine	64
B Economic Formulas	65
C Contract Reference	66
D Event Guide	67
E Security Invariants	68
F Supported Asset Model	69
G Glossary	70
H References	71

How to Read This Whitepaper

This document explains one piece of software: the raffle.fun smart contracts as they exist at Git commit `a2120f5`, together with the applications built around them. It is written for two readers at once.

The first reader has never used a blockchain. Every chapter's main text is written for you: each new idea gets a definition, an everyday comparison, and a concrete example before anything technical appears, and unfamiliar terms are defined where they first appear or in the glossary in Appendix G.

The second reader is technical: a developer, auditor, investor, or integration partner. For you, the precise details live in the "Under the hood" boxes, the tables, the figures, and the appendices. Appendix A holds the exact state machine, Appendix B the exact arithmetic, Appendix C the contract reference, and Appendix E the security invariants. Nothing in an appendix contradicts the main text; it only sharpens it.

The recurring boxes

The same few labeled boxes appear throughout the document, and they always mean the same thing:

- **In one sentence:** the shortest true summary of the chapter.
- **Example:** the running fictional raffle, described below.
- **Why this matters:** the design motivation in plain language.
- **What the contract enforces:** rules that hold because code enforces them onchain.
- **What this does not guarantee:** the honest edge of each guarantee.
- **Under the hood:** implementation detail for technical readers, safe to skip.
- **Important risk:** something that can cost a user money.
- **For sponsors** and **For ticket holders:** advice specific to one role.

The running example

One fictional raffle appears throughout the book so that every rule can be shown with real numbers. Ava, an artist, raffles a one-of-one digital artwork called *Sunset Study #7*: tickets cost 10 USDC, the minimum is 100 tickets, the sale runs 14 days, and Ben, Maya, Leo, and Noor buy in. Depending on the chapter, the sale ends with 120 tickets sold, with 80, or with none. All example arithmetic was recomputed with the protocol's own math library; none of it is rounded by hand.

What this document is not

This whitepaper is not marketing and not advice. It describes unaudited, undeployed software at one specific commit, including its limitations and failure modes. Where a statement could not be verified against the code, it was left out; the companion file `docs/whitepaper/FACT-CHECK.md` maps each significant claim to its source.



PART I

The Idea

What raffle.fun is, the problem it answers, and what "onchain" really buys you.

1 Executive Summary

2 raffle.fun in 60 Seconds

3 The Problem

4 The Core Idea

5 What "Onchain" Means

Executive Summary

raffle.fun is a protocol for running prize raffles where the prize is a digital collectible (an NFT) and every rule is enforced by a program on a public blockchain rather than by a person. This chapter compresses the whole document into two pages.

What is raffle.fun?

raffle.fun lets anyone whose payment token is on an approved list run a provably fair raffle for a single NFT. The person giving away the prize, called the sponsor, locks the NFT into a small, single-purpose program called a raffle contract. That contract, not the sponsor and not the raffle.fun team, then holds the prize, sells the tickets, holds the money, runs the draw, and pays everyone out according to rules that were fixed the moment the raffle was created.

The software consists of onchain contracts (a factory that creates raffles, the raffle logic itself, and a read-only helper), plus supporting tools: a web application, a developer kit, and an indexer that makes raffle history searchable. The contracts target Ethereum, the largest smart-contract network, and Base, a low-fee network built on Ethereum technology. At the commit this document describes, the contracts are complete and tested but not yet deployed to any public network, and they have not been independently audited.

Who is it for?

Three groups. Sponsors: artists, collectors, brands, and communities who want to distribute an NFT by chance instead of by auction or fixed-price sale. Participants: anyone who wants a transparent, equal-odds chance at that NFT for the price of a ticket. Builders: developers who want raffle mechanics they can integrate without trusting an operator, because every rule is public and enforced by code.

How does a raffle work?

A sponsor creates a raffle by choosing the prize NFT, a payment token from the factory's approved list (for example USDC, a digital dollar), a ticket price, a minimum number of tickets, a sale window of up to 30 days, and a recovery address that can reclaim the prize in every ending where the winner does not take it. The prize moves into the contract in the same transaction. Buyers then purchase tickets at the fixed price. Each ticket is itself a small digital asset with equal odds, and tickets can be transferred to other people while the sale runs.

When the sale closes, anyone may pay a small fee to ask Pyth Entropy, an independent randomness service, for one random number. That number picks exactly one winning ticket. If at least the minimum number of tickets was sold, the winning ticket's holder claims the NFT, the sponsor is credited 95 percent of the ticket money, and the protocol treasury is credited a 5 percent fee. If fewer

than the minimum sold, the winner instead is credited 76 percent of the pot in cash, the sponsor is credited 19 percent, the treasury still receives 5 percent, and the prize returns to the recovery address.

If the draw never happens, the protocol does not strand anyone. When no draw request succeeds within 3 days of closing, or a requested draw is unanswered for 2 days, anyone can flip the raffle into a refund mode in which every ticket is credited its exact purchase price back, no fee is charged, and the recovery address reclaims the prize. Every payout in every ending is pull-based: the contract records what each person is owed, and they withdraw it themselves whenever they choose. Claims never expire.

What does the administrator control?

Remarkably little, and nothing inside a live raffle. The factory owner, expected to be a multisig (an account requiring multiple signatures), can admit or remove payment tokens for future raffles, change the treasury address for future raffles, and pause the creation of new raffles. It cannot touch an existing raffle's prize, money, rules, deadlines, winner, or refunds, because no function exists that would let it.

The most important risks

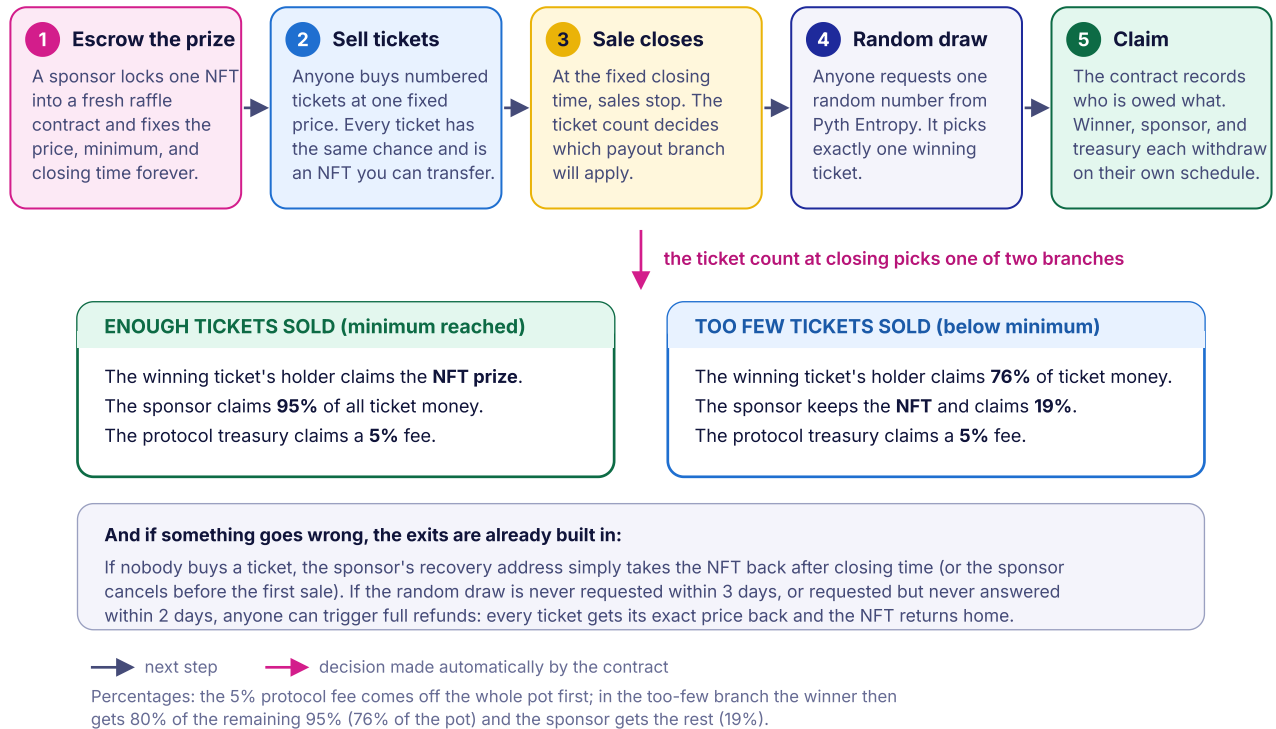
The software is unaudited, and testing cannot prove the absence of bugs. The prize NFT can be counterfeit or worthless: the contract escrows whatever the sponsor deposits and cannot judge authenticity. Payment tokens are real-world instruments whose issuers may freeze accounts. Normal settlement depends on the Pyth randomness service; if it stalls, the outcome is a full refund rather than a prize. Chance-based prize distribution is regulated differently across jurisdictions, and this document makes no compliance claims. Chapter 33 lists everything the protocol cannot promise.

CHAPTER 2

raffle.fun in 60 Seconds

This page is the whole protocol in one picture. If you read nothing else, read this.

FIGURE 1 raffle.fun at a glance



The five steps of every raffle, the two payout branches the ticket count selects, and the built-in exits for every way a raffle can end without a draw. Percentages are fixed constants compiled into the contract: a 5 percent protocol fee, then in the cash branch an 80/20 split of the remainder between winner and sponsor.

IN ONE SENTENCE

A sponsor locks one NFT in a contract that sells equal-odds tickets, has an independent service pick one winning ticket, and lets everyone withdraw exactly what the fixed rules say they are owed, with full refunds if the draw never happens.

CHAPTER 3

The Problem

Prize raffles are ancient and popular, but running one fairly requires the operator to make promises that participants cannot verify. This chapter describes the general weaknesses of manually run raffles. They are design problems inherent to trusting an operator, not accusations against any specific business.

Opaque custody

In a conventional online raffle, the operator holds the prize and the money at the same time. Participants cannot see whether the prize actually exists, whether it is already promised to someone else, or whether ticket revenue is kept separate from the operator's own funds. If the operator disappears mid-raffle, both prize and payments usually go with them.

Rules that can change

The price, the deadline, the number of tickets, and the payout split live in a database the operator edits at will. Extending a sale that is going well, quietly adding tickets, or changing the split after money has arrived are all one keystroke away, and participants would rarely be able to tell.

Unclear odds

"One winner will be chosen from all entries" says nothing about how many entries exist. Without a public count of tickets sold, a participant cannot know whether they hold one chance in fifty or one in fifty thousand, and the operator can inflate the count invisibly.

Discretionary winner selection

A drawing performed on a spreadsheet, a stream, or a website widget is a drawing the operator can redo until it likes the answer. Even honest operators cannot prove a single, final, untampered draw; dishonest ones can pick a friend and produce a plausible-looking ceremony.

Settlement risk

Winning is not the same as being paid. The winner of a manual raffle receives the prize if and when the operator sends it. Refunds after a failed or cancelled raffle arrive if and when the operator processes them. Every step depends on the operator's solvency, honesty, and continued existence.

A single point of failure

One website, one database, one bank account, one person with the password. If any of these fails, is hacked, or is seized, the raffle and its assets fail with it.

WHY THIS MATTERS

Every one of these weaknesses is a promise a human must keep. The design question behind raffle.fun is: how many of those promises can be replaced with a program whose behavior anyone can inspect and no one can quietly change?

The Core Idea

raffle.fun moves the raffle itself, custody, rules, tickets, accounting, drawing, and payouts, into a smart contract: a program deployed on a public blockchain that runs exactly as written and cannot be edited afterward.

What moves into the contract

The prize sits inside the contract from creation until the outcome is settled, so it provably exists and provably cannot be sold twice. The rules (price, minimum, deadlines, splits, recovery address) are fixed at creation; there is no edit function. The tickets are numbered digital assets minted by the contract itself, so the total count is public at every moment and the advertised odds are checkable. The money accumulates inside the contract, and the payout logic that eventually distributes it is the same public code everyone inspected before buying. The draw comes from an external randomness service through a single authenticated channel, and the contract accepts exactly one result. Finally, the failure paths are part of the program: fixed deadlines convert a missing draw into full refunds without asking anyone's permission.

What stays outside the contract

Honesty about the boundary matters as much as the boundary itself. The contract cannot verify that an NFT is authentic or valuable, only that a specific token from a specific contract was deposited. It cannot control the payment token's issuer, who may be able to freeze balances. It does not generate randomness itself; it verifies the source and consumes one result. The website people use is replaceable infrastructure, not the raffle. And no code can decide whether running a raffle is lawful where you live.

IN ONE SENTENCE

raffle.fun replaces the promises a raffle operator makes with rules a program enforces, and is explicit about the few promises no program can enforce.

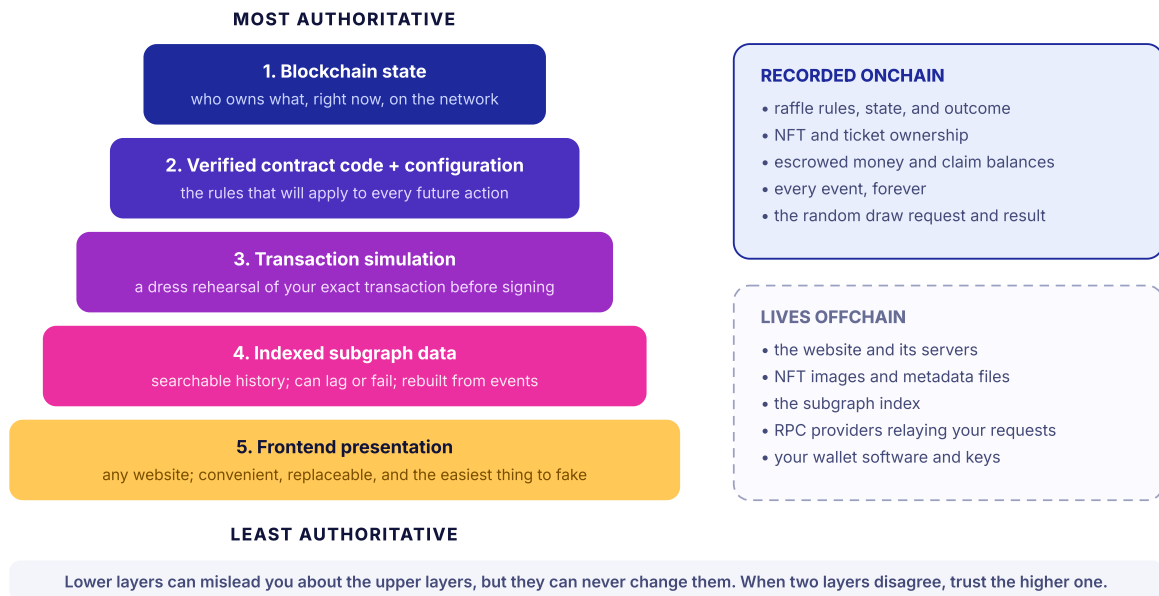
What "Onchain" Means

"Onchain" is shorthand for "recorded in a blockchain's shared ledger, where everyone sees the same data and no single party can rewrite it." This chapter separates what lives onchain from what merely talks about it, because the difference decides what you can safely trust.

A blockchain is a public record book kept simultaneously by thousands of computers that continuously check each other. Ethereum and Base, the networks raffle.fun targets, are such ledgers: they store account balances, program code, and program state, and they execute transactions, which are signed instructions submitted by users. A wallet is the app that holds your signing key and submits those transactions; gas is the small fee the network charges to process one.

For a raffle, the ledger holds everything that decides who gets what: the raffle's rules and current stage, who owns the prize, who owns each ticket, how much money the contract holds and who may claim it, the stored randomness request, and a permanent log of events. Around that core sits convenience infrastructure that lives offchain: the raffle.fun website, the images and descriptions attached to NFTs, the subgraph (a searchable index rebuilt from onchain events), the RPC providers that relay your requests to the network, and your wallet software itself.

FIGURE 2 The hierarchy of authority



Five layers, from the blockchain state at the top to websites at the bottom. Each lower layer is more convenient and less authoritative. A compromised website can lie about the chain, but it cannot change the chain; the defense is checking important facts against a higher layer before signing.

IMPORTANT RISK

Most real-world losses in blockchain applications happen at the bottom two layers: a fake or hacked website presents false information, and a user signs a transaction based on it. The raffle.fun web application re-reads contract state and simulates every transaction immediately before asking a wallet to sign, but no page can protect a user who signs whatever a hostile page requests. When the stakes are high, verify the raffle's address and rules against the chain itself.



PART II

The People and the Assets

Who does what in a raffle, and exactly which digital assets are at stake.

6 Participants

7 The Prize NFT

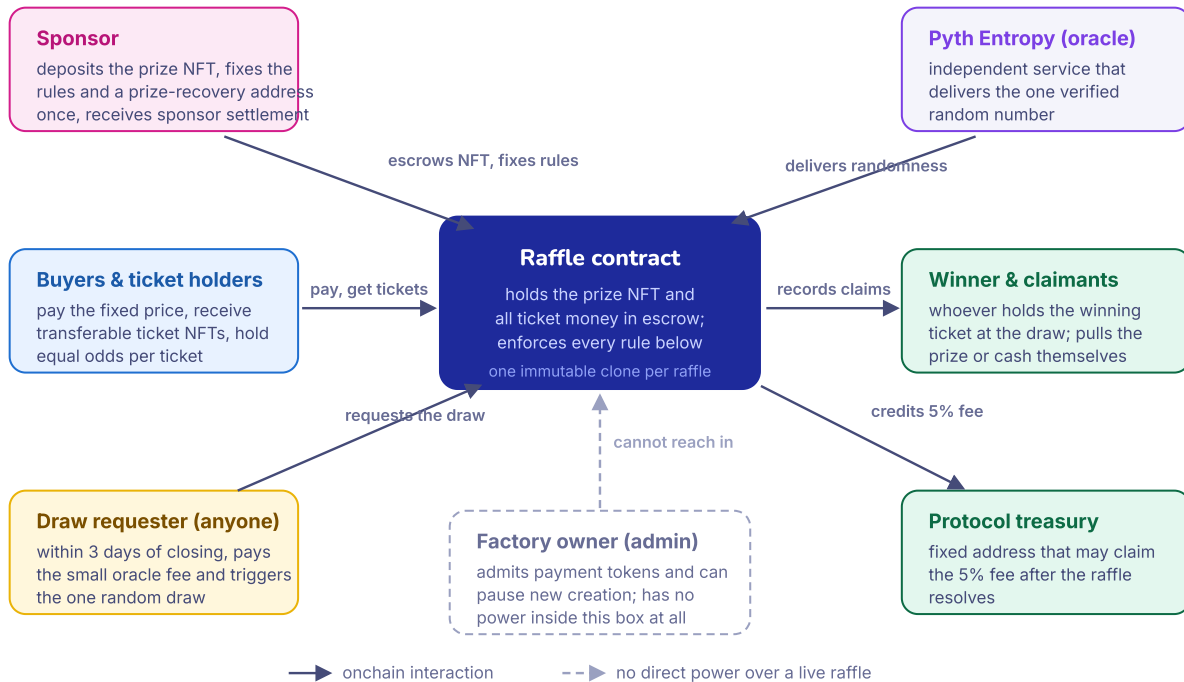
8 The Payment Token

9 Ticket NFTs

Participants

A raffle involves a handful of roles. One person can hold several of them at once (the sponsor can buy tickets; a buyer can gift tickets to a friend), but each role has exactly defined powers.

FIGURE 3 Participant and role map



Everyone who touches a raffle, arranged around the raffle contract that enforces the rules between them. The dashed border marks the one party with no path into a live raffle: the protocol administrator.

The **sponsor** creates the raffle, deposits the prize, and fixes every rule in one transaction, including a **recovery address** where the prize returns in each ending where the winner does not take it (by default, the sponsor itself). After the first ticket sells, the sponsor has no special powers left except receiving its own settlement. The **buyer** pays for tickets; the **ticket recipient** is whoever the buyer names to receive them, which enables gifts. The **current ticket holder** is whoever owns a ticket at a given moment; tickets are transferable assets, so this can change. The **winner** is the holder of the winning ticket at the instant the random result arrives, recorded permanently. The **draw requester** is any person who pays the small randomness fee after the sale closes; this is a public service anyone can perform, as is the **refund finalizer** role that opens and credits refunds after a failed draw. The **protocol treasury** is a fixed address credited the 5 percent fee when a draw resolves a raffle. The **factory owner** administers only the creation of future raffles. **Pyth Entropy** is the external randomness service. The **frontend and indexer** display and organize; they hold no power.

PARTICIPANT	CAN DO	CANNOT DO	MUST TRUST
Sponsor	create the raffle, fix all rules, cancel while zero tickets sold, claim its settlement	change any rule after creation, cancel after a sale, pick the winner, touch the pot	that enough buyers find the raffle attractive
Buyer	buy 1 to 100 tickets per transaction at the fixed price, direct tickets to any recipient	buy outside the sale window, pay a different price, exceed contract-level checks	the prize's authenticity, the payment token's issuer
Ticket holder	hold or transfer tickets while transfers are open, win, claim refunds for tickets held at failure	transfer during the draw freeze or before an uncredited refund, change odds except by holding more tickets	the same external parties as buyers
Winner	claim the NFT or cash exactly as the branch dictates, choose a safe destination	claim twice, claim someone else's share, be skipped by anyone	that they keep control of their own wallet keys
Draw requester	trigger the one draw within its 3-day window by paying the oracle fee	influence the random result, request twice	Pyth to answer; else refunds open
Refund finalizer	flip a failed raffle into refund mode after its deadline, credit refund batches	choose who is refunded or how much, take a cut	nothing; the action is mechanical
Protocol treasury	claim its 5 percent fee after a resolved draw	charge more, charge on refunds or cancellations, intervene anywhere	nothing inside the raffle
Factory owner	admit or remove payment tokens for future raffles, set future treasury, pause new creation, hand over ownership	touch any existing raffle in any way	its own key security (a multisig is expected)
Pyth Entropy	deliver the one random result for the stored request	forge a request, deliver twice, redirect funds	its own cryptographic and operational design
Frontend / indexer	display, search, and simplify	change state, block claims, alter outcomes	nothing; users should verify against the chain

CHAPTER 7

The Prize NFT

IN ONE SENTENCE

The prize is a specific token from a specific NFT contract, locked inside the raffle from the moment of creation until exactly one authorized claim releases it.

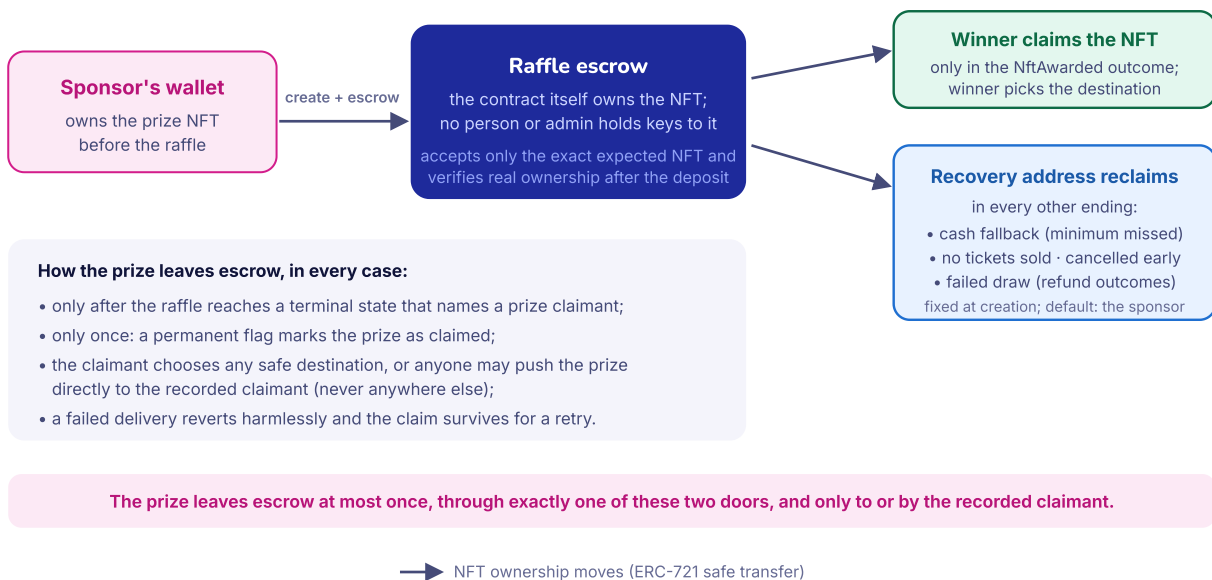
An **NFT** (non-fungible token) is a blockchain record that says "token number N of collection C belongs to address A." The standard that defines how such tokens behave on Ethereum-style chains is called **ERC-721**. Owning an NFT means controlling the key of the address the ledger lists as its owner; nothing more, nothing less.

Escrow means handing an asset to a neutral keeper until an outcome decides who gets it. Here the keeper is the raffle contract itself. In the same transaction that creates the raffle, the factory moves the prize from the sponsor's wallet into the new raffle contract. From that moment, the sponsor cannot sell, move, or reclaim the NFT except through the raffle's own endings.

How the contract validates the prize

The raffle refuses everything except the exact expected deposit: it must be in its initial waiting state, the token must come from the configured collection, carry the configured token ID, come from the sponsor, and be delivered by the factory. After the transfer, the factory additionally asks the NFT contract who owns the token now and requires the answer to be the raffle. A collection that lies about ownership fails this check and the whole creation reverts, sponsor unharmed.

FIGURE 4 Prize custody flow



The prize's whole life: one road in, and exactly two doors out. The winner's door opens only in the `NftAwarded` outcome; the recovery address's door opens in every other ending. The claim can be retried if a delivery fails, and anyone may push the prize to the recorded claimant, but never anywhere else.

IMPORTANT RISK

Escrow proves possession, not authenticity or value. A counterfeit "copy" of a famous artwork is a perfectly valid ERC-721 token, and the contract will escrow it faithfully. Some NFT collections are also upgradeable or administratively controlled, meaning their own operator could freeze or alter tokens while they sit in escrow. Buyers must judge the collection itself; Chapter 36 shows how.

The Payment Token

IN ONE SENTENCE

All ticket money in a raffle is denominated in one ERC-20 token, chosen by the sponsor from a short list the factory owner has reviewed and admitted.

An **ERC-20 token** is the standard for interchangeable blockchain money: units are identical, divisible, and held as balances. USDC, a token redeemable one-to-one for US dollars, is the typical example. In raffle.fun this is called the **quote token** because it is the unit the ticket price is quoted in.

Internally, token amounts are integers in the token's smallest unit. USDC has six decimal places, so "10 USDC" is stored as the raw integer 10,000,000. Every formula in this document operates on such raw units, which is why rounding, when it happens, is always a matter of a single indivisible unit (a millionth of a dollar for USDC).

Why an allowlist?

ERC-20 is a loose standard, and hostile or merely unusual tokens can break naive accounting: some skim a fee from every transfer, some rebase balances up and down, some return misleading values. The raffle defends itself mechanically: on every purchase it measures its own balance before and after and requires the increase to equal the price exactly, and on every payout it verifies both its own decrease and the recipient's increase. A token that fails these checks makes transactions revert rather than corrupting accounts. On top of that, the factory only allows raffle creation with tokens its owner has reviewed and admitted (at most 32 at a time). Removing a token later stops new raffles from using it but changes nothing for raffles that already exist.

WHAT THIS DOES NOT GUARANTEE

The allowlist and the exact-transfer checks cannot remove issuer power. A token like USDC has a blacklist function: if the issuer blacklists a claimant's address, that claimant's withdrawal will fail while everyone else's still works, and the claim simply waits (it can also be sent to a different destination the claimant chooses, or the token issuer may unblock the address). If an issuer pauses the entire token, all withdrawals in that token wait until it resumes. The raffle cannot print replacement money.

Ticket NFTs

IN ONE SENTENCE

Every ticket is itself an ERC-721 token with a sequential number, equal odds, and normal transferability except during two well-defined freezes.

When Ben pays 50 USDC for 5 tickets in Ava's raffle, the contract mints him tickets numbered, say, 6 through 10. Making tickets real tokens rather than database rows has three consequences. First, the supply is provable: `totalTickets` is public state, so "you hold 5 of 87 tickets" is a verifiable statement, and the contract even exposes a convenience view that computes your current odds. Second, tickets are property: you can transfer one to a friend, sell it, or hold it in another wallet you control, all before the draw is requested. Third, the winner is determined by ownership at a precisely defined instant, not by whoever's name is on a list.

Numbering and odds

Tickets are numbered from 1 upward with no gaps; each purchase mints a contiguous block to the chosen recipient. The draw picks one number in that range with equal probability for every ticket, including ticket 1 and the last ticket sold. Holding ten tickets gives exactly ten times the chance of holding one; there are no weighted, discounted, or bonus tickets.

When transfers freeze

Transfers work normally while the raffle is Active, including after the sale closes but before a draw is requested. The moment a draw request succeeds, transfers freeze so that nobody can trade tickets while the random result is in flight. If the raffle resolves normally, transfers reopen immediately, but by then the winner has been recorded permanently, so the tickets become transferable souvenirs whose movement changes nothing. If the raffle instead fails into refund mode, each individual ticket stays frozen until its refund has been credited to its holder, and then becomes a souvenir too. Chapter 16 walks through the timing in detail.

FOR TICKET HOLDERS

Buying a ticket for someone else? Purchases take a recipient address, so the ticket can be minted directly to your friend's wallet. Whoever holds the ticket at the draw, or at refund crediting, gets the winnings or the refund. Gift accordingly.



PART III

How a Raffle Works

The full mechanics, from the creation transaction to the moment the winner is fixed.

10 Creating a Raffle

11 Buying Tickets

12 The Raffle Timeline

13 The Minimum-Ticket Threshold

14 Requesting the Draw

15 How Randomness Works

16 Freezing the Winner

Creating a Raffle

IN ONE SENTENCE

One transaction creates the raffle contract, fixes every rule forever, and locks the prize inside it; if any part fails, nothing happens at all.

Ava wants to raffle Sunset Study #7. She first approves the raffle.fun factory to move that one NFT (a standard ERC-721 permission). Then she calls the factory's createRaffle function with her chosen parameters. Everything in the following table is fixed permanently the moment the transaction succeeds.

INPUT	MEANING	RULES THE FACTORY ENFORCES
Prize contract and token ID	which exact NFT is at stake	must be a deployed contract that affirmatively reports ERC-721 support
Payment token	the one currency of this raffle	must be on the factory's admitted list at creation time
Recovery address	where the prize returns in every non-winner ending	zero means "use the sponsor"; fixed forever
Ticket price	gross price per ticket, in raw token units	must be nonzero; what you advertise is what buyers pay
Minimum tickets	the threshold that picks the payout branch	must be nonzero; not a sales cap
Start time	when sales open (inclusive)	now, or up to 7 days in the future; zero means "now"
End time	when sales stop (exclusive)	after start; the sale window may be at most 30 days
Metadata URI	a link describing the raffle, shown on every ticket	at most 2,048 bytes

Ava picks: 10 USDC per ticket, minimum 100 tickets, a 14-day sale starting now, and her own vault wallet as the recovery address.

What the one transaction does

The factory checks every input, deploys a fresh raffle contract at a predictable address, initializes it with the full configuration, registers it so applications can recognize official raffles, emits a creation event that indexers use, transfers the prize from Ava into the new contract, and finally verifies with the NFT contract itself that the raffle now owns the prize. Atomicity is the point: there is no state in which a raffle exists without its prize, or a prize left a wallet without its raffle. If Ava forgot the NFT approval, or the collection misbehaves, the entire transaction reverts.

UNDER THE HOOD

The new raffle is an EIP-1167 minimal proxy (a "clone"): a tiny contract that delegates all logic to one shared, locked implementation while keeping its own isolated storage. Its address is computed with CREATE2 from the chain ID, the raffle's sequence number, the sponsor, the payment token, and the exact prize, so integrators can predict it before creation. The clone accepts initialization only from a factory whose recorded implementation matches the one baked into the clone's own bytecode, which blocks look-alike factories. Ticket collections are named "Raffle Fun Ticket #N" with symbol "RFT-N".

CHAPTER 11

Buying Tickets

IN ONE SENTENCE

Buying is a two-step ritual, approve then buy, and the contract checks the exact payment, the exact window, and the exact quantity before minting numbered tickets.

Because ERC-20 tokens require a spending permission, Ben first approves the raffle to take 50 USDC, then calls `buyTickets` asking for 5 tickets for himself. Wallets that support transaction batching can present both steps as one confirmation; the `raffle.fun` web app uses that when available and falls back to two steps when not. That app behavior is convenience, not protocol: any wallet that can call the contract can buy tickets.

The contract enforces, in order: the raffle is Active with the prize in escrow; the time is within the sale window (start inclusive, end exclusive); the recipient is a real address; the quantity is between 1 and 100; and the total price, ticket price times quantity, does not overflow. It then pulls the payment and verifies its own balance grew by exactly that amount, which is what rejects fee-skimming or otherwise dishonest tokens. Only then does it mint the tickets: a contiguous block of new numbers to the recipient Ben named, which can be himself or anyone else. One event records buyer, recipient, quantity, ticket range, and amount paid.

If anything fails, everything fails: a reverted purchase leaves no half-minted tickets and takes no money. There is no per-person limit beyond the 100-per- transaction bound; a determined buyer can send multiple transactions, and each added ticket dilutes everyone's odds visibly, including theirs.

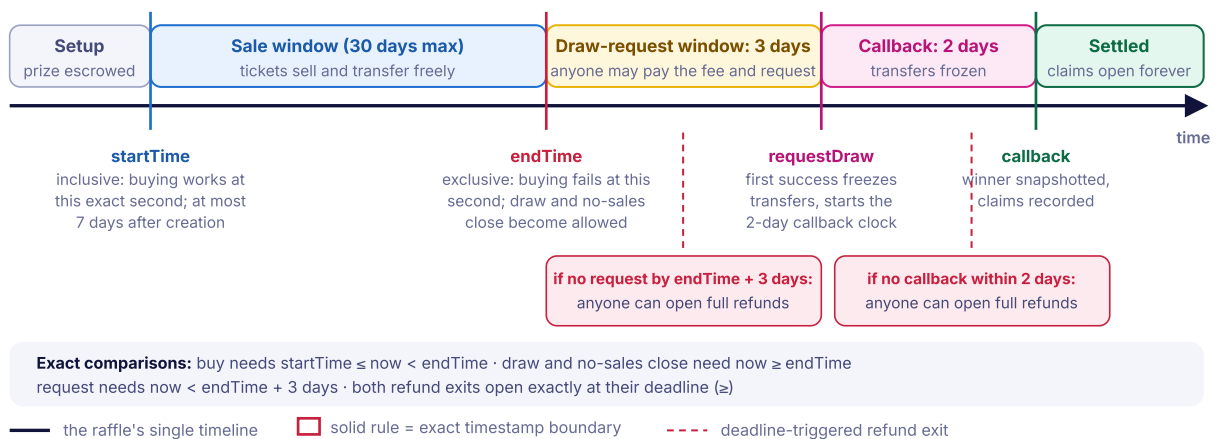
EXAMPLE

Maya buys 10 tickets for 100 USDC and directs 2 of them in a second purchase to her friend Leo's wallet. The pot now holds Ben's 50, Maya's 100, and Leo's tickets cost Maya 20. Nothing about a later purchase changes what an earlier buyer paid, but every purchase changes the live odds, which the contract exposes for anyone to read.

The Raffle Timeline

Every raffle lives on a single, fixed timeline. Two figures carry this chapter: the clock view and the state-machine view of the same journey.

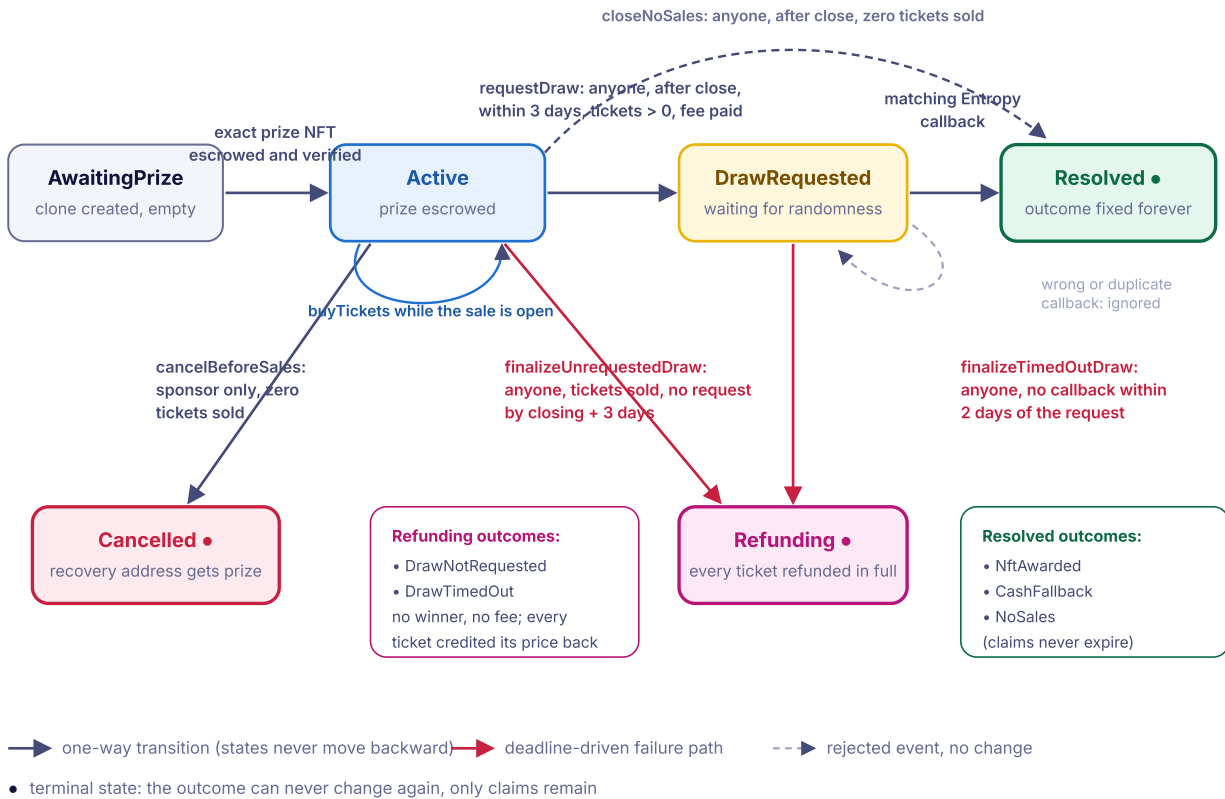
FIGURE 5 The raffle clock



The phases of a raffle in time order, with every boundary the contract compares against. Solid vertical rules are exact timestamps; the dashed drops beneath show the two deadline-triggered refund exits. The sale window boundaries are chosen by the sponsor within fixed caps; the 3-day and 2-day windows after it are protocol constants.

The boundary semantics are exact and worth stating once, precisely. Buying requires $\text{start} \leq \text{now} < \text{end}$: a purchase in the very second the sale ends already fails. Closing actions (the draw request, and the no-sales close) require $\text{now} \geq \text{end}$: they work in that same second. The draw request additionally requires $\text{now} < \text{end} + 3 \text{ days}$. The two failure exits open at their deadlines inclusively: missing-request finalization at $\text{end} + 3 \text{ days}$, callback-timeout finalization 2 days after the request was accepted.

FIGURE 6 The complete state machine



Every state a raffle can occupy and every transition between them, with the exact caller and condition on each arrow. Red arrows are the deadline-driven failure paths anyone can trigger. Dots mark terminal states: once reached, only claims remain, and nothing can ever change the outcome.

A raffle is created in **AwaitingPrize** and becomes **Active** in the same transaction once the prize is verified. From **Active** it can move four ways: to **Cancelled** (sponsor cancels while zero tickets are sold), to **Resolved** via the no-sales close (anyone, after the end, zero tickets), to **DrawRequested** (anyone requests the draw in its window), or to **Refunding** (the request window expires unused). From **DrawRequested** it moves to **Resolved** when the matching random result arrives, or to **Refunding** if the result stays missing for 2 days. No transition ever runs backward.

CHAPTER 13

The Minimum-Ticket Threshold

IN ONE SENTENCE

The minimum is a promise about outcomes, not a cap on sales: it decides which payout branch runs, while sales continue to the fixed closing time regardless.

Think of the minimum as the sponsor saying: "this prize is worth giving away only if at least this much interest shows up." Ava set 100 tickets at 10 USDC, an implied 1,000 USDC of gross interest. The threshold is evaluated exactly once, inside the draw's callback, by one comparison: tickets sold $\geq$ minimum.

SCENARIO AT THE DRAW	COMPARISON	BRANCH
99 tickets sold	$99 \geq 100$ is false	cash fallback: winner gets 76 percent of the pot, prize returns
100 tickets sold	$100 \geq 100$ is true	NFT awarded: equality counts as met
101 tickets sold	$101 \geq 100$ is true	NFT awarded; sales had continued normally

Sales do not stop at 100. If the raffle is popular, tickets keep selling until the closing time; 120 tickets mean a 1,200 USDC pot. Each ticket past the minimum grows the pot and dilutes everyone's individual odds, visibly, in real time. There is also no upper limit on what a sponsor may set as a minimum. An unrealistically high minimum cannot break accounting; it simply makes the cash-fallback branch the likely outcome, which buyers can see before buying.

FOR TICKET HOLDERS

Before buying, look at two numbers together: tickets sold so far versus the minimum (how likely is the NFT branch?) and tickets sold overall (what are my odds?). The official app shows both, plus the flip point where the outcome would switch.

CHAPTER 14

Requesting the Draw

IN ONE SENTENCE

After the sale closes, anyone, not just the sponsor, can pay a small fee in ETH to request the raffle's one and only random draw within a 3-day window.

Randomness is not free: Pyth Entropy charges a fee, denominated in the network's native currency (ETH), to fund the callback transaction it will send later. The raffle exposes the current fee, read live from Entropy for the raffle's configured callback gas allowance. Whoever calls `requestDraw` attaches at least that fee; the contract forwards the exact fee to Entropy and credits any overpayment to the caller as a withdrawable balance, so overpaying by a safety margin is harmless.

Making the request permissionless removes a hostage scenario: no single party, not the sponsor and not the operator of any website, is the mandatory trigger. Any ticket holder who wants the draw to happen can make it happen for the cost of the fee. The request must satisfy: raffle Active, sale ended, at least one ticket sold, and now within 3 days after the end. The first success is also the last: the state advances before the external call, so a second request has nothing to act on. If the request transaction reverts (say, the fee reading failed), nothing changed and anyone can simply try again within the window.

UNDER THE HOOD

requestDraw stores the exact timestamp (starting the 2-day callback clock), sets an in-flight guard, and only then calls Entropy's requestV2 with the configured gas limit. The returned sequence number, the unique ID of this randomness request, is stored; the guard closes. A hostile "callback" delivered synchronously during the request, before the sequence is stored, is ignored by the guard. The DrawRequested event carries the sequence, requester, fee paid, refundable excess, request time, and the callback deadline, so indexers can display the full schedule.

WHAT THIS DOES NOT GUARANTEE

The protocol cannot force anyone to request the draw. If 3 days pass with no successful request, an unlikely event for any raffle with interested participants, since any of them can do it, the raffle becomes finalizable into full refunds, as Chapter 21 describes.

CHAPTER 15

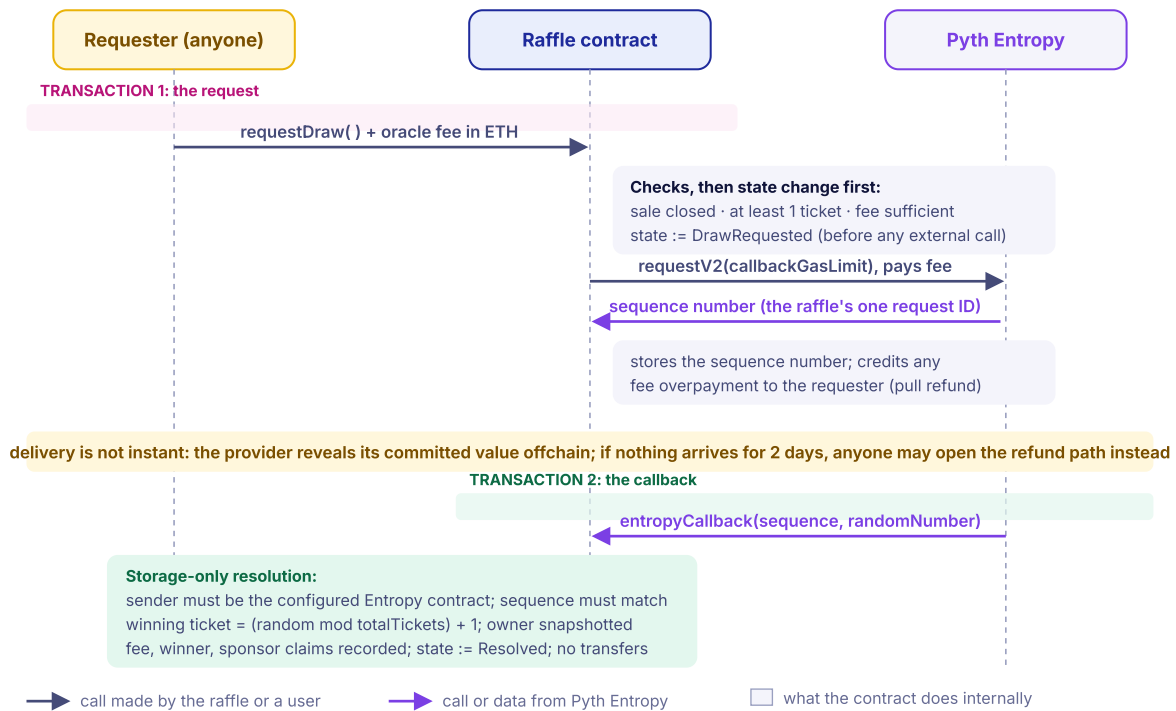
How Randomness Works

IN ONE SENTENCE

The raffle asks an independent service, Pyth Entropy, for one random number through an authenticated channel, and turns it into a winning ticket with one formula anyone can recheck.

Why not just use something already onchain, like a block's timestamp? Because everything onchain is either predictable or influenceable by the parties who order transactions, and an administrator picking a number is exactly the discretion this protocol exists to remove. An **oracle** is a service that brings information from outside the chain onto it; **Pyth Entropy** is an oracle specialized in randomness. It works on a commit-reveal principle: parties commit to hidden values in advance and the revealed combination is unpredictable to each of them alone.

FIGURE 7 The draw, transaction by transaction



Two separate transactions: the request anyone sends, and the callback only the Entropy contract can deliver. Between them lies an offchain wait. The contract's checks on the callback are shown in the green box; everything it does is a storage write, never an asset transfer.

From random number to winning ticket

The callback delivers one 256-bit random number. The winning ticket is:

```
winningTicketId = (randomNumber mod totalTickets) + 1
```

The modulo operation maps the huge random number onto the range 1 through totalTickets, inclusive on both ends. A raffle with one ticket always selects ticket 1; the final ticket is exactly as likely as any other. The contract then reads that ticket's current owner and records them as the winner, permanently.

UNDER THE HOOD

Strictly speaking, reducing a 256-bit number modulo N is perfectly uniform only when N divides 2 to the 256th power. Otherwise some residues occur once more than others across the full input space, giving a relative bias of at most N divided by 2 to the 256th. For any realistic ticket count (even billions), that bias is far below one in 10 to the 50th, which is many orders of magnitude smaller than, say, the probability of guessing a private key. The whitepaper therefore says "equal odds" without qualification in the main text, and notes the exact caveat here.

Authentication and replay

The callback path is locked down three ways: the caller must be the configured Entropy contract (enforced by Pyth's consumer wrapper), the raffle must be in DrawRequested, and the delivered sequence number must equal the stored one. Anything else, a duplicate delivery, a wrong sequence, a delivery after the raffle already settled, is ignored with a logged event and zero state change. One result is accepted, ever. If delivery is delayed, Pyth's own tooling can replay the same sequence; a replay delivers the same stored request and therefore the same result. What the raffle does about prolonged silence is Chapter 21's subject: after 2 days, the refund path opens.

WHAT THIS DOES NOT GUARANTEE

This design makes the draw verifiable and non-repeatable, but its unpredictability rests on Pyth's cryptographic and operational assumptions holding. The protocol chose a specialist randomness service over homemade alternatives precisely because those assumptions are explicit, documented, and independently scrutinized. They are still assumptions, and Chapter 31 places them on the trust map.

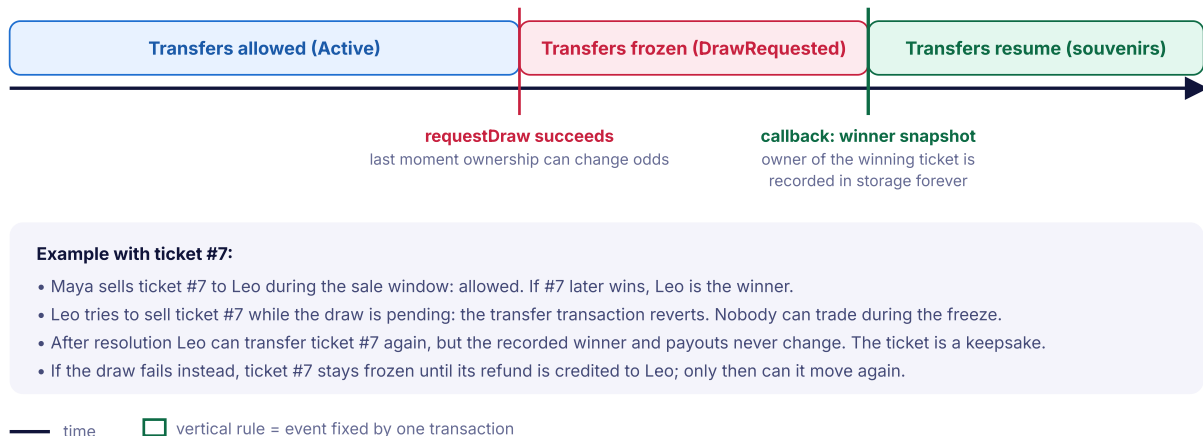
CHAPTER 16

Freezing the Winner

IN ONE SENTENCE

Ticket ownership matters up to one precise instant per path: the draw callback for winnings, and refund crediting for refunds; transfers are frozen in the run-up to each so the snapshot cannot be gamed.

FIGURE 8 Ticket transfers and the winner snapshot



The three transfer regimes of a normal raffle: free movement while Active, a freeze from the successful draw request until resolution, and souvenir transfers afterward. The example beneath follows one ticket through a sale, the freeze, and both possible endings.

The rules, stated once, exactly:

- While the raffle is Active, tickets transfer freely. Selling a ticket transfers the chance it represents: if it later wins, the new holder is the winner.
- The successful draw request freezes all owner-to-owner transfers. Nobody can trade while the random result is in flight, so nobody can buy a win after the fact or dump a losing ticket in the window between request and result.
- The callback snapshots the winning ticket's owner into permanent storage. From then on, transfers reopen and tickets are souvenirs; moving one moves no claim.
- On the failure path, entering Refunding freezes each ticket until its refund is credited to its current holder. The credited holder is thereby exactly the person who bore the loss of the failed raffle. After crediting, that ticket too becomes a freely transferable souvenir.

EXAMPLE

Maya transfers ticket 7 to Leo two days before the sale ends: allowed, and if ticket 7 wins, Leo wins. During the draw freeze Leo tries to sell it: the transaction reverts with a transfers-frozen error. The draw picks ticket 87 (Noor's), so after resolution Leo's ticket 7 is a keepsake he can move anywhere. In the alternate failure timeline, ticket 7 stays locked until the refund batch credits Leo his 10 USDC, and only then can the ticket move.



PART IV

Outcomes and Money

Every way a raffle can end, and where every unit of money goes in each one.

17 Outcome Overview

18 Threshold Met: the NFT Branch

19 Threshold Missed: the Cash Branch

20 No Sales and Early Cancellation

21 Failed Draws, Timeouts, and Refunds

22 Fees and Accounting

23 Why Claims Are Pulled

Outcome Overview

Every raffle ends in exactly one of six recorded outcomes. This chapter is the map; the next four chapters walk each road.

FIGURE 9 All six terminal outcomes

RESOLVED BY THE RANDOM DRAW

NftAwarded When: tickets $\geq$ minimum NFT: winner claims it Winner: the NFT, no cash Sponsor: 95% of the pot Fee: 5% of the pot Refunds: none Finalized by the Entropy callback	CashFallback When: tickets < minimum NFT: recovery address reclaims Winner: 76% of the pot in cash Sponsor: 19% of the pot + rounding Fee: 5% of the pot Refunds: none Finalized by the Entropy callback	How to read these cards "Pot" is every ticket payment held by the raffle. Percentages come from the fixed constants: a 5% protocol fee, then in the cash branch 80% of the remainder to the winner (76% of the pot) and the rest to the sponsor (19% plus rounding dust). "Recovery address" is the prize-return destination the sponsor fixed at creation (by default, the sponsor itself).
--	---	---

NO DRAW EVER HAPPENS

NoSales / CancelledBeforeSale When: zero tickets ever sold NFT: recovery address reclaims Money: none ever existed Fee: none Draw: never requested NoSales: anyone closes after endTime. Cancelled: sponsor only, before sale 1.	DrawNotRequested When: no draw request succeeds within 3 days of closing NFT: recovery address reclaims Tickets: full price refunded, each ticket exactly once Fee: none Anyone can finalize after the deadline and credit refunds in batches	DrawTimedOut When: a request was accepted but no callback within 2 days NFT: recovery address reclaims Tickets: full price refunded, each ticket exactly once Fee: none A late callback still wins if it lands before the timeout transaction
--	---	---

Every raffle ends in exactly one of these six recorded outcomes. All payouts and refunds are pull-based claims that never expire.

The two outcomes decided by the random draw (top) and the four endings where no draw ever completes (bottom). Every card lists who gets the NFT, who gets money, and who may finalize. No other endings exist in the contract.

OUTCOME	TRIGGERED BY	NFT GOES TO	TICKET HOLDERS RECEIVE	SPONSOR RECEIVES	PROTOCOL FEE	RANDOMNESS	FINALIZED BY
NftAwarded	draw lands, tickets $\geq$ minimum	winner	nothing (winner got the NFT)	95% of pot	5% of pot	required	Entropy callback
CashFallback	draw lands, tickets < minimum	recovery address	winner alone: 76% of pot	19% of pot plus rounding dust	5% of pot	required	Entropy callback
NoSales	end passes, zero sold	recovery address	no holders exist	nothing (no pot)	none	never requested	anyone

OUTCOME	TRIGGERED BY	NFT GOES TO	TICKET HOLDERS RECEIVE	SPONSOR RECEIVES	PROTOCOL FEE	RANDOMNESS	FINALIZED BY
CancelledBeforeSale	sponsor cancels, zero sold	recovery address	no holders exist	nothing (no pot)	none	never requested	sponsor only
DrawNotRequested	3 days pass after end, no request	recovery address	full ticket price back, per ticket	nothing	none	never requested	anyone
DrawTimedOut	request accepted, 2 days no callback	recovery address	full ticket price back, per ticket	nothing	none	requested, unanswered	anyone

Three regularities are worth noticing. The prize always goes to exactly one of two places: the winner (only in `NftAwarded`) or the fixed recovery address (everywhere else). The protocol charges its fee only when a verified random draw actually resolved the raffle; every failure and every empty raffle is free. And the two refund outcomes are economically identical: what differs is only which deadline expired.

CHAPTER 18

Threshold Met: the NFT Branch

IN ONE SENTENCE

When the draw lands with at least the minimum sold, the winning ticket's holder claims the NFT, the sponsor is credited 95 percent of the pot, and the treasury 5 percent.

Ava's raffle closes at 120 tickets, comfortably past her minimum of 100. Noor holds ticket 87. Someone requests the draw; the random number arrives and maps to ticket 87. In that single callback transaction the contract records: winner Noor, outcome `NftAwarded`, prize claimant Noor, treasury credited 60 USDC (5 percent of the 1,200 pot), sponsor credited the remaining 1,140 USDC. No tokens move yet.

Noor then calls `claimPrize` with the destination of her choice, and the NFT leaves escrow to that address. Ava calls `claimQuote` and withdraws 1,140 USDC; the treasury withdraws its 60. Each claim is independent. If Noor is on vacation for a month, her claim waits untouched; nothing expires, and nobody can jump the queue or redirect it. Alternatively, anyone may call the push variants (`claimPrizeFor`, `claimQuoteFor`) which deliver only to the recorded claimant, useful for helping a less technical winner.

The winner receives no share of the money in this branch; the winner's prize is the NFT itself. All other ticket holders receive nothing, which is what "raffle" means; their tickets remain as souvenirs.

Threshold Missed: the Cash Branch

IN ONE SENTENCE

When the draw lands below the minimum, the raffle still draws one winner, but the prize is cash: 80 percent of the post-fee pot, while the NFT returns to the sponsor's recovery address.

Suppose instead only 80 tickets sold by closing time, and the draw picks ticket 12, held by Ben. The pot is 800 USDC. The callback computes: fee 40 USDC (5 percent), distributable 760, winner's share 608 (80 percent of 760, which is 76 percent of the whole pot), sponsor's share the exact remainder, 152. The prize claimant is the recovery address Ava configured. Ben withdraws 608 USDC; Ava withdraws 152 and her vault wallet reclaims the artwork.

The design reasoning: the sponsor set the minimum as the level of interest below which parting with the prize is not worth it, so the prize returns. But buyers took a real chance and someone must win something real, so the bulk of the pot becomes a cash prize. The sponsor's 19 percent compensates the effort of running the raffle while keeping the sponsor's incentive pointed at reaching the minimum, not missing it. Ticket holders other than the winner receive nothing in this branch either.

Rounding, stated plainly: both the fee and the winner's share round down to the smallest token unit, and the sponsor's share is defined as what remains, so the three parts always sum to the pot exactly. With USDC the largest possible rounding effect is under two millionths of a dollar, always in the sponsor's favor.

No Sales and Early Cancellation

Two quiet endings share a property: no money ever entered, so nothing needs distributing and neither branch involves randomness or fees.

No sales. The closing time passes with zero tickets sold. Anyone (a bot, an indexer, Ava herself) calls `closeNoSales`; the raffle resolves with outcome `NoSales`, and the recovery address may reclaim the NFT. Because the call is permissionless, a sponsor who lost their keys after creation does not strand the prize: anyone can close, and the recovery address (a separate wallet, if the sponsor chose wisely) can reclaim.

Early cancellation. Before the first ticket sells, and only then, the sponsor may cancel outright, at any moment, even mid-sale or after the end. The raffle moves to the Cancelled state, and the recovery address reclaims the prize. The instant ticket 1 sells, this power is gone forever: from then on the raffle can only end through a draw, a no-sales close (impossible once a ticket exists), or the refund path. A sponsor can therefore never pull the prize out from under paying participants.

WHAT THE CONTRACT ENFORCES

cancelBeforeSales checks the caller is the sponsor and that totalTickets is zero; closeNoSales checks the end has passed and totalTickets is zero. Both set the prize claimant to the fixed recovery address and are terminal. There is no path in the contract that cancels a raffle with even one sold ticket.

CHAPTER 21

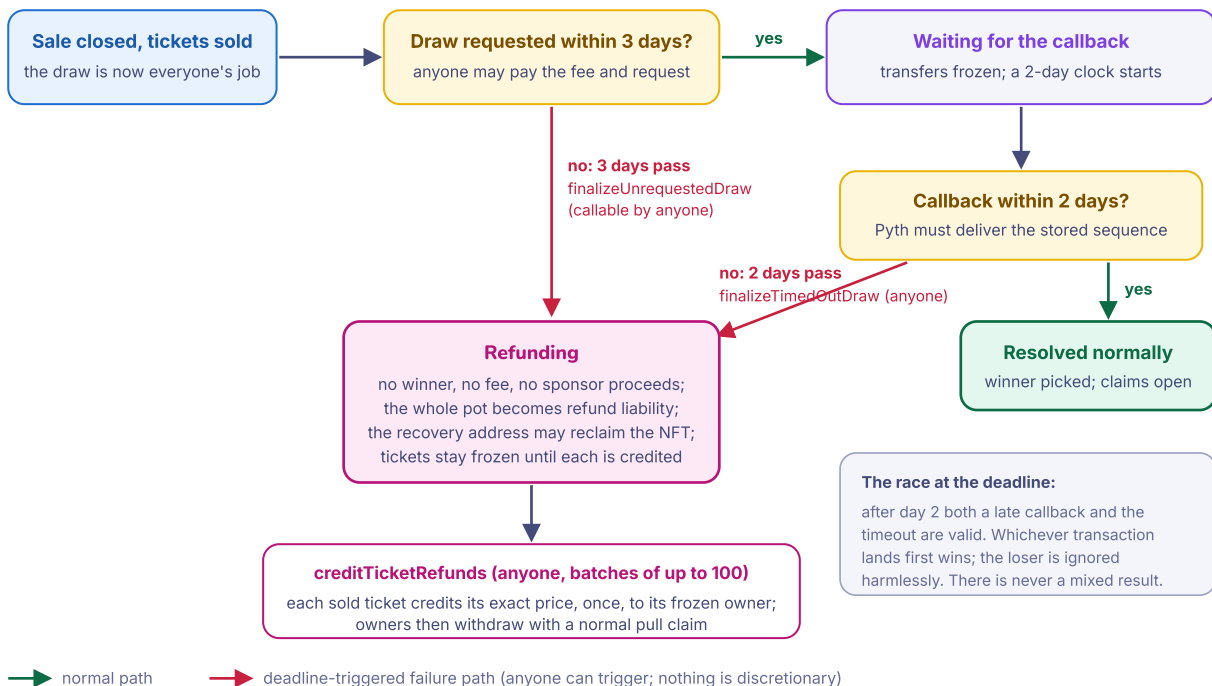
Failed Draws, Timeouts, and Refunds

IN ONE SENTENCE

If the draw is never requested within 3 days of closing, or a requested draw goes unanswered for 2 days, anyone can switch the raffle into refund mode: every ticket is credited its exact price back, no fee is charged, and the prize returns home.

A raffle that sold tickets owes its participants an ending. Normally that ending is the draw; but the draw depends on two things the contract cannot compel: some person sending the request, and the oracle answering it. The protocol converts both dependencies into fixed deadlines with a deterministic exit.

FIGURE 10 Timeout and refund flow



The two questions every sold-out-of-time raffle answers: was the draw requested in 3 days, and did the callback arrive in 2? Two 'no' roads lead to the same Refunding state, where anyone can credit every ticket its refund. The gray box explains the race a late callback can still win.

The two deadlines

The request window: from the closing time until 3 days after it, `requestDraw` is open to everyone. At the deadline, `requestDraw` shuts and `finalizeUnrequestedDraw` opens, also to everyone. The callback window: an accepted request starts a 2-day clock; if no valid callback lands in time, `finalizeTimedOutDraw` opens to everyone. One subtlety is deliberate: a late callback is still honored until the moment a timeout transaction actually executes. At the boundary the two race, the first one included on the chain wins, and the loser becomes a harmless no-op. What can never happen is a mixed result: either the raffle resolved with a winner, or it refunds everyone, never both.

How refunds work

Entering Refunding does three things at once: the entire pot is reclassified as refund debt, the prize claimant becomes the recovery address, and outcome `DrawNotRequested` or `DrawTimedOut` is recorded. No protocol fee is charged, and the sponsor receives none of the pot: a failed raffle earns nobody anything.

Refunds are then credited ticket by ticket, in permissionless batches of up to 100 ticket IDs per transaction. For each ticket, the contract records the current holder, marks the ticket as credited (each ticket exactly once, enforced), and adds exactly one ticket price to that holder's withdrawable balance. Crediting touches no external contract, so a hostile holder cannot block a batch that includes them, and one holder's problem never delays another's refund. Because each ticket is frozen from the moment Refunding begins until it is credited, the refund goes precisely to whoever held the ticket when the raffle failed. Holders then withdraw with the same pull claim as any other payout; the batching cost is borne by whoever volunteers to send the crediting transactions (typically the frontend operator or any interested holder), never deducted from refunds.

EXAMPLE

The 80-ticket raffle's request window expires with no request (say the fee spiked and everyone hesitated). Anyone calls `finalizeUnrequestedDraw`. The 800 USDC pot becomes 80 refunds of exactly 10 USDC. One transaction credits tickets 1 through 80: Ben's balance rises by 50 (his 5 tickets), Maya's by 80, Leo's by 20 for the two tickets Maya gave him (the holder, not the payer, is refunded), Noor's by 200, and so on. Ava's vault reclaims the artwork. Nobody profits and nobody loses except the 3 days everyone spent waiting.

WHAT THIS DOES NOT GUARANTEE

The refund guarantee covers what the contract controls: the money is reserved, the crediting is permissionless, and the claims never expire. It does not cover a payment-token issuer freezing transfers, a chain outage outlasting every deadline, or a holder losing their own keys. Those sit on the trust map in Chapter 31.

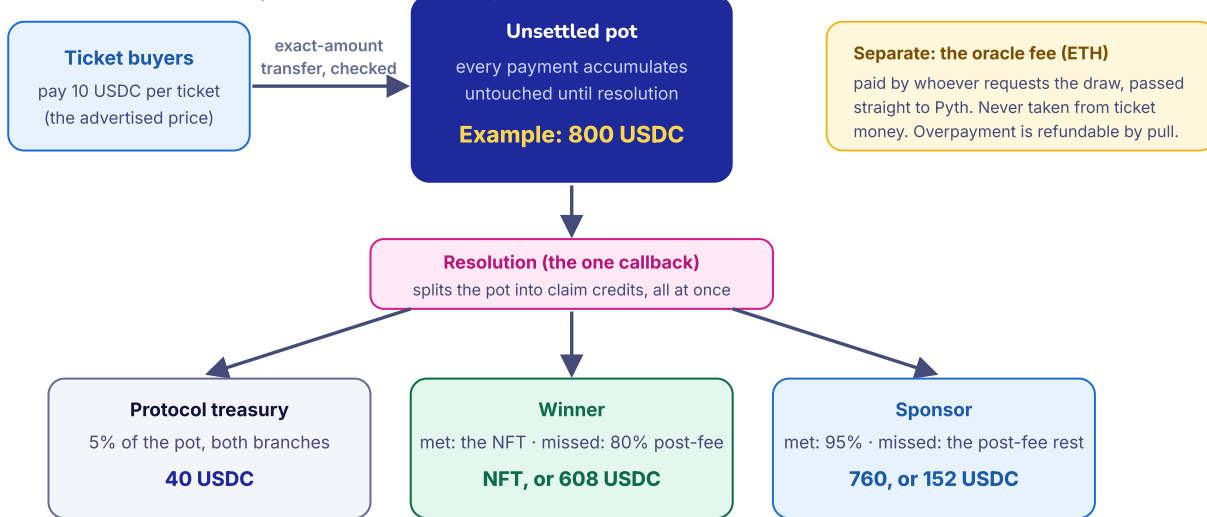
Fees and Accounting

IN ONE SENTENCE

One fee exists in the whole protocol: 5 percent of the pot, charged once, only when a verified random draw resolves the raffle; every other flow is conservation.

FIGURE 11 Where the money goes

Amounts follow the worked example: 80 tickets sold at 10 USDC, minimum 100. Arrows show quote-token movement or credit.



Conservation: fee + winner cash + sponsor cash = the exact pot. Example (missed): 40 + 608 + 152 = 800 USDC. Nothing is lost or minted.

Failed draw instead? The split never happens: the whole 800 USDC becomes refunds of exactly 10 USDC per ticket, and there is no fee.

The flow of ticket money from buyers through the pot to the three possible claimants, with the worked example's exact numbers, and the two flows that never mix with it: the oracle fee in ETH, and the failed-draw case where the split never happens and the full pot becomes refunds.

The formulas, exactly as implemented

All amounts are raw integer token units; divisions round down (floor).

purchase	gross = ticketPrice × quantity
resolution	fee = floor(pot × 500 / 10,000)
	distributable = pot − fee
threshold met	sponsorCash = distributable (winner takes the NFT)
threshold missed	winnerCash = floor(distributable × 8,000 / 10,000)
	sponsorCash = distributable − winnerCash
failed draw	refund per ticket = ticketPrice (fee = 0)
identity	fee + winnerCash + sponsorCash = pot (resolved branches)
	sum of refunds = pot (refund branches)

Two design choices deserve a sentence each. The fee is computed once on the aggregate pot at resolution, not per purchase, so splitting a purchase into many tiny ones cannot shave the fee through repeated rounding. And the ticket price is gross: the number advertised is exactly the number a buyer pays, with the fee taken from the pot later rather than added on top at checkout.

Worked examples, verified against the code

CASE	POT	FEE (5%)	WINNER	SPONSOR
120 tickets x 10 USDC, minimum met	1,200.00	60.00	the NFT	1,140.00
80 × 10 USDC, minimum missed	800.00	40.00	608.00	152.00
99 × 10 USDC, one below minimum	990.00	49.50	752.40	188.10
10 × 0.333333 USDC, missed (dust case)	3.333330	0.166666	2.533331	0.633333
1 × 10 USDC, minimum missed	10.00	0.50	7.60	1.90
any failed draw	pot	0	0	0; all pot refunded

The dust case shows floor rounding at work: the exact 5 percent of 3,333,330 raw units is 166,666.5, which floors to 166,666, and the winner's 80 percent floors likewise; the sponsor's remainder absorbs both half-units. Every row reproduces exactly with the SDK's math functions and matches the contract's own test vectors.

The solvency invariant

At every moment, the raffle's actual token balance is at least the sum of its three liabilities: the unsettled pot, the not-yet-credited refund debt, and the total of all recorded claims. Tokens sent to the raffle by mistake (a direct donation rather than a purchase) sit above that line as visible surplus: they are never counted as anyone's claim, can never trigger or alter a settlement, and have no withdrawal path. The same discipline applies to the native currency: only draw-fee overpayments are claimable, and force-pushed ETH is inert surplus. These properties are enforced by construction and continuously checked by the invariant test suite.

CHAPTER 23

Why Claims Are Pulled

IN ONE SENTENCE

The contract never sends assets to anyone on its own initiative; it records debts, and each claimant withdraws, which turns one shared settlement into many independent small transactions that cannot block each other.

The alternative, pushing every payout automatically inside the callback, fails in predictable ways. A single recipient who cannot receive tokens (a contract wallet that rejects transfers, a blacklisted address) would make the entire settlement revert, holding everyone else hostage. Pushing also maximizes reentrancy exposure, where a malicious recipient's code runs in the middle of the sender's bookkeeping. And it spends the oracle's limited callback gas on arbitrary third-party code. The pull pattern, an application of the checks-effects-interactions principle, avoids all three: the callback writes numbers and exits.

What pulling means in practice:

- Independence: your claim is a separate transaction. Anyone else's failure, malice, or absence cannot delay it.
- Retryability: a claim that fails (say, your chosen destination rejects the NFT) reverts atomically; the claim survives, and you retry with a better destination.
- Destination choice: `claimQuote` and `claimPrize` let the claimant direct assets to any safe address they control, which matters if their usual address is, for example, blacklisted by a token issuer. The one forbidden destination is the raffle itself.
- Help without power: the claim-for variants let anyone pay the gas to deliver a claim, but only to the recorded claimant, never to a destination of the helper's choosing.
- No deadlines: claims persist indefinitely. There is no sweep, no expiry, and no administrator who could confiscate an unclaimed balance.

UNDER THE HOOD

Every state-changing function is wrapped in a reentrancy guard, and every claim clears the stored balance before touching the token, then verifies the raffle's exact debit and the recipient's exact credit; a token that misdelivers reverts the whole claim, restoring the debt. The prize claim sets its claimed flag before the transfer for the same reason. These checks are what turn "a token behaved oddly" into "the transaction reverted harmlessly" rather than "the books are wrong."



PART V

Architecture

The contracts and tools that make up raffle.fun, and which of them you actually have to trust.

24 The Contract System

25 Architecture Diagram

26 What Is Authoritative?

27 Deployment and Versioning

The Contract System

raffle.fun is deliberately small: three production contracts onchain, and three supporting tools off it.

RaffleFactory is the front door. It creates raffles, enforces every creation rule (admitted payment token, valid times, real contracts, verified escrow), keeps the registry that marks which raffle addresses are genuine, and holds the three administrative dials described in Chapter 30. It never holds user assets.

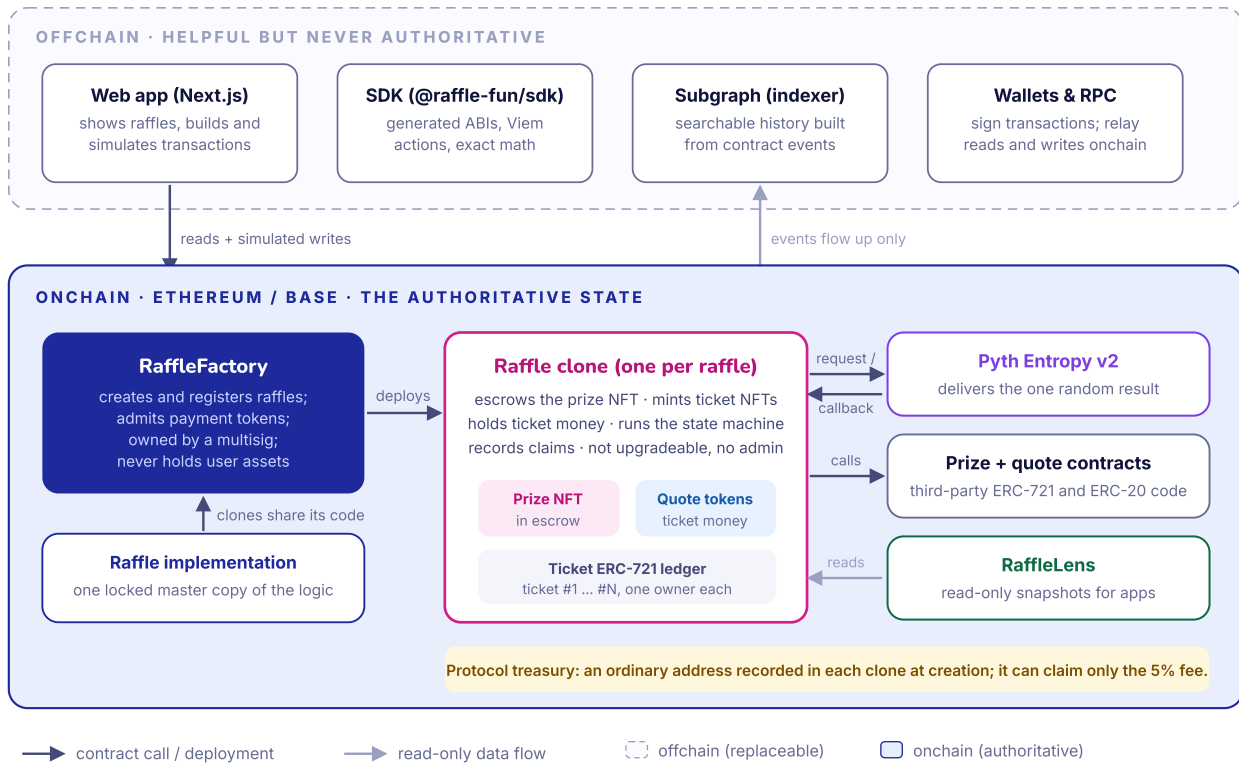
Raffle is the heart: one contract per raffle, holding that raffle's prize, money, tickets, and state machine. Each raffle is an **EIP-1167 clone**: a tiny proxy that borrows its logic from a single locked implementation contract while keeping fully separate storage. Cloning gives every raffle identical, auditable behavior at a fraction of the deployment cost, and total isolation: a thousand raffles share code but nothing else, so no raffle's assets are ever exposed to another's activity. Crucially, these clones are not upgradeable proxies: the implementation address is fixed in the clone's bytecode, the implementation itself is locked against initialization, and no function anywhere can swap logic later.

RaffleLens is a stateless convenience: one call returns a raffle's full state, deadlines, liabilities, and what actions a given wallet could take right now. It refuses addresses the factory has not registered, so applications built on it cannot be tricked into rendering a counterfeit raffle. It can only read.

Offchain, the **SDK** (a TypeScript library) carries the contract interfaces and exact-math helpers so integrators compute the same numbers the contracts do; the **subgraph** indexes events into a searchable history; and the **web app** is the official interface. All three are conveniences: the protocol is complete without them.

Architecture Diagram

FIGURE 12 The complete system



Everything in one view: the authoritative onchain layer (factory, locked implementation, per-affle clones, the external token contracts, Pyth Entropy, and the read-only lens) and the replaceable offchain layer (web app, SDK, subgraph, wallets). Arrows are labeled with the only kinds of interaction that exist between the parts.

Reading the diagram top to bottom: users act through wallets and the web app, which talk to the chain via RPC. Writes go to the factory (creation) or a specific raffle clone (everything else). The clone holds the two escrowed assets and talks to exactly one external service, Pyth Entropy, over its request-and-callback channel. Events flow up from the chain into the subgraph, which feeds discovery and history back to the app: a one-way street. The lens serves batched reads. Nothing in the offchain layer can write state, and no onchain component depends on the offchain layer to function.

What Is Authoritative?

When two sources of information disagree, which one is right? The protocol's answer is a strict hierarchy, and it is worth internalizing because most practical attacks are attempts to make you act on a lower layer's lie.

1. **Blockchain state:** the ledger itself. Who owns the prize, who holds which ticket, what every balance is. This is the ground truth.
2. **Verified contract code and configuration:** the published source that matches the deployed bytecode, and the raffle's fixed parameters. This is what will happen to the state.
3. **Transaction simulation:** a dry run of your exact transaction against current state, which the official app performs before every signature request.
4. **The subgraph:** indexed history. Fast and searchable, but it can lag the chain by blocks or fail entirely, and it is rebuilt, not original.
5. **Frontend presentation:** any website, including the official one. Maximally convenient, minimally authoritative, and the only layer an attacker can fake cheaply.

The web application is built to respect this hierarchy: lists and history come from the subgraph, but every fact that matters to a transaction (price, state, claims, verification status, the entropy fee) is re-read live from the chain immediately before the wallet is asked to sign, and the transaction is simulated first. A compromised or counterfeit frontend can still mislead a user into signing something harmful, which is why the safety checklists in Chapter 38 begin with verifying addresses.

Deployment and Versioning

At the reviewed commit, the contracts are **not deployed to any network**. The repository deliberately contains no addresses: its deployment registry holds only a validation schema, and the tooling refuses to fabricate or copy addresses from anywhere else. When a deployment happens, it follows a documented runbook: deploy implementation, factory, and lens; verify the source on the block explorer; transfer factory ownership to a multisig via a two-step handshake; record every address and the exact source commit in a validated deployment file; and run a full end-to-end raffle on the Base Sepolia test network before any production use. The runbook's non-negotiable gates include an independent security review and legal review, and mainnet deployment is intentionally excluded from the automated tooling.

Versioning is by replacement, never mutation. A future protocol version means a new implementation and a new factory deployed beside the old ones. Existing raffles continue on the code they were created with, forever; nothing about a new version reaches back. An application may point users at the new factory, but that is a frontend choice, visible onchain as a different factory address.

WHAT THE CONTRACT ENFORCES

Clone configuration is immutable after initialization: no setter functions exist on a raffle. The implementation contract permanently disables its own initialization. The factory's constants (fee, splits, deadlines, bounds) are compiled in, so even the factory owner cannot alter them without deploying an entirely new factory, which cannot affect existing raffles.



PART VI

Security and Trust

What the protocol promises, how the code defends those promises, and what remains outside its power.

28 Security Goals

29 Security Design Choices

30 Administrator Powers

31 The Trust and Dependency Map

32 Threat Model

33 What the Protocol Cannot Guarantee

34 Testing and Review

Security Goals

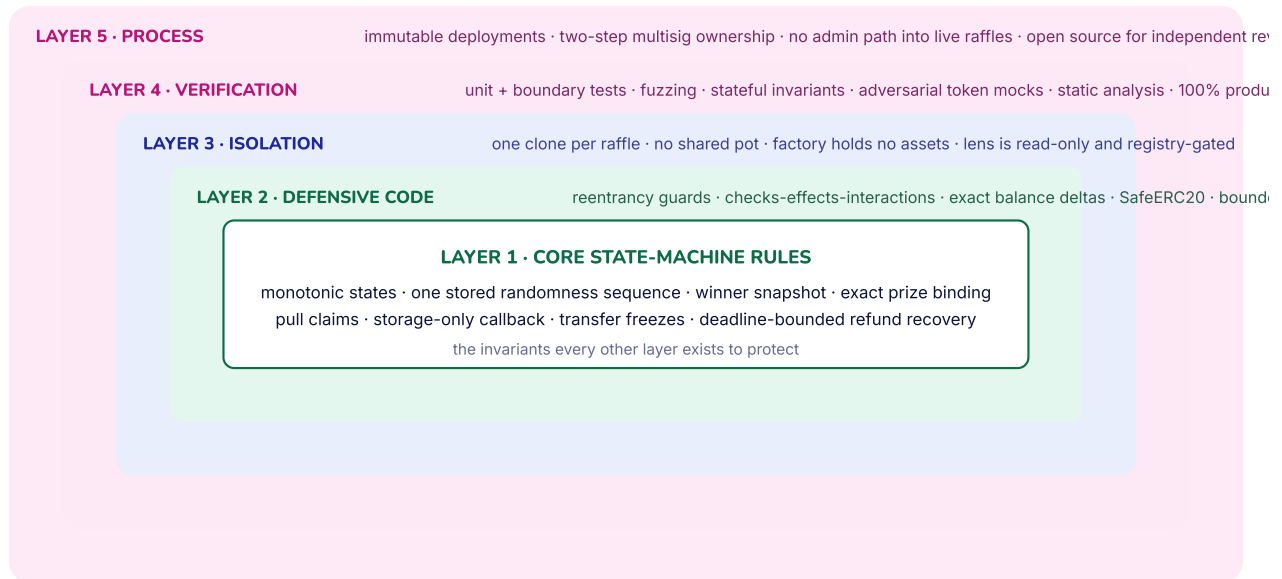
The protocol's security posture reduces to a short list of promises, each of which is enforced by code and continuously attacked by the automated test suite. In plain language:

- **The prize leaves escrow at most once**, and only to or by the one recorded claimant of the raffle's terminal outcome.
- **Exactly one resolution**: a raffle accepts one randomness sequence and one terminal settlement; nothing re-rolls, re-draws, or overwrites a result.
- **Every sold ticket can win**: the selection formula covers ticket 1 through the last ticket, inclusively, with equal probability.
- **The books always balance**: the contract's token balance always covers the pot, the uncredited refunds, and every recorded claim; claims can be spent once.
- **A failed draw refunds everyone exactly once**: each sold ticket credits its exact price to its failure-time holder, and the refund pool equals the pot.
- **Donations change nothing**: assets pushed at the contract outside its flows are inert surplus, never a trigger, never anyone's claim.
- **The administrator cannot reach into a live raffle**: no function exists by which the factory owner touches an existing raffle's assets, rules, or outcome.
- **Recovery is bounded**: every liveness dependency (the draw request, the callback) has a fixed deadline with a permissionless exit.

Appendix E states each of these formally alongside the invariant tests that check them.

Security Design Choices

FIGURE 13 Defense in depth



An attacker must defeat every layer from the outside in. Each layer assumes the ones outside it may fail. None of this equals an independent audit.

Five concentric layers protect the core state-machine rules. An attack must penetrate process controls, the verification suite, architectural isolation, and the defensive coding layer before it can even engage the core invariants. Each layer assumes the ones outside it may fail.

The notable choices, and why each was made:

Non-upgradeable clones with a locked implementation. Upgradeability is a backdoor with good intentions. Removing it converts "trust the team not to change the rules" into "the rules cannot change." The cost, that bugs cannot be hot-fixed either, is treated as the price of the promise and mitigated by testing and by versioned replacement (Chapter 27).

Factory-only initialization with mutual authentication. A clone accepts configuration only from a factory that provably serves the same implementation the clone runs, closing off look-alike factory attacks.

Exact escrow validation. The prize receiver binds contract, token ID, sender, operator, and state; the factory then re-verifies real ownership. Unrelated NFTs bounce; dishonest collections fail creation.

Exact balance deltas on every token movement, in and out, with SafeERC20 wrappers. Fee-skimming, rebasing, and misreporting tokens produce clean reverts, not corrupted books.

Reentrancy guards plus checks-effects-interactions everywhere. Storage is settled before any external contract is invoked, and guarded entry points make nested calls revert. The adversarial test suite includes reentrant tokens, a reentrant prize collection, and a reentrant ticket receiver.

A storage-only oracle callback. The randomness callback writes state and stops. It transfers nothing and calls no third-party code, so it cannot be grieved by gas or by hostile recipients, and its gas use is measured in tests with a required safety margin under the configured limit.

Bounded work in every transaction. Purchases mint at most 100 tickets; refund batches credit at most 100 tickets; lens reads batch at most 100 raffles; metadata is capped. No loop's length is attacker-controlled without a bound.

Monotonic state with idempotent rejection. States move forward only; wrong, stale, or duplicate callbacks are logged and ignored; double finalization is structurally impossible.

Transfer freezes at the two decisive moments (draw pending, refund pending), so ownership snapshots cannot be manipulated mid-decision.

Two-step, multisig-ready ownership. Factory ownership transfers require the recipient to accept, preventing loss to a typo; the deployment runbook requires the final owner to be a multisig.

CHAPTER 30

Administrator Powers

Trust claims should be falsifiable, so this chapter is the complete inventory of administrative power at the reviewed commit, derived from the code rather than from policy statements.

FIGURE 14 The administrator's exact powers

The factory owner CAN ...	The factory owner CANNOT ...
● change the treasury address that future raffles capture ● admit or remove up to 32 payment tokens for new raffles a hard creation gate; removal never touches existing raffles ● pause the creation of new raffles at this factory ● hand ownership to another address (two-step transfer) ● deploy a new factory version alongside this one Blast radius if the owner key is stolen: future raffles could capture a thief's treasury, labels could mislead discovery, and creation could be paused. Every existing raffle keeps running exactly as configured.	✗ change any existing raffle's price, minimum, dates, fee, or split ✗ pick, replace, or influence a winner, or request a second draw ✗ seize or redirect the prize, the pot, or anyone's claim ✗ upgrade, pause, cancel, or settle an existing raffle ✗ change an existing raffle's token, recovery address, or deadlines ✗ block buying, claiming, refunds, or the draw on a live raffle Why: these functions do not exist in the code. A raffle clone has no owner role, no upgrade hook, no pause switch, and no rescue function. The limits are structural, not a policy promise that could quietly change later.

Derived from RaffleFactory.sol access control and the absence of admin functions in Raffle.sol at the reviewed commit.

Everything the factory owner can do, everything it cannot, and the blast radius if its key were stolen. The cannot-side holds because the functions do not exist in the deployed code, not because of a promise.

The factory owner **can**: admit or remove payment tokens for future raffle creation (a bounded list of at most 32); change the treasury address that future raffles will capture; pause the creation of new raffles; and transfer its own role via a two-step handshake. Each of these affects only raffles that do not yet exist.

The factory owner **cannot**: modify any existing raffle's price, minimum, times, deadlines, token, recovery address, fee, or split; select, replace, or influence a winner; trigger, block, or redo a draw; pause, cancel, upgrade, or settle an existing raffle; seize or redirect any prize, pot, refund, or claim; or extend its own reach, because raffles contain no owner role and no function that consults the factory after initialization.

A compromised owner key is therefore an inconvenience, not a catastrophe: it could pause new creation, poison the future-raffle token list, redirect future (not current) fee capture, or hand ownership to the attacker, and all of it is publicly visible onchain the moment it happens. Every raffle already running, and every claim inside it, would continue exactly as before.

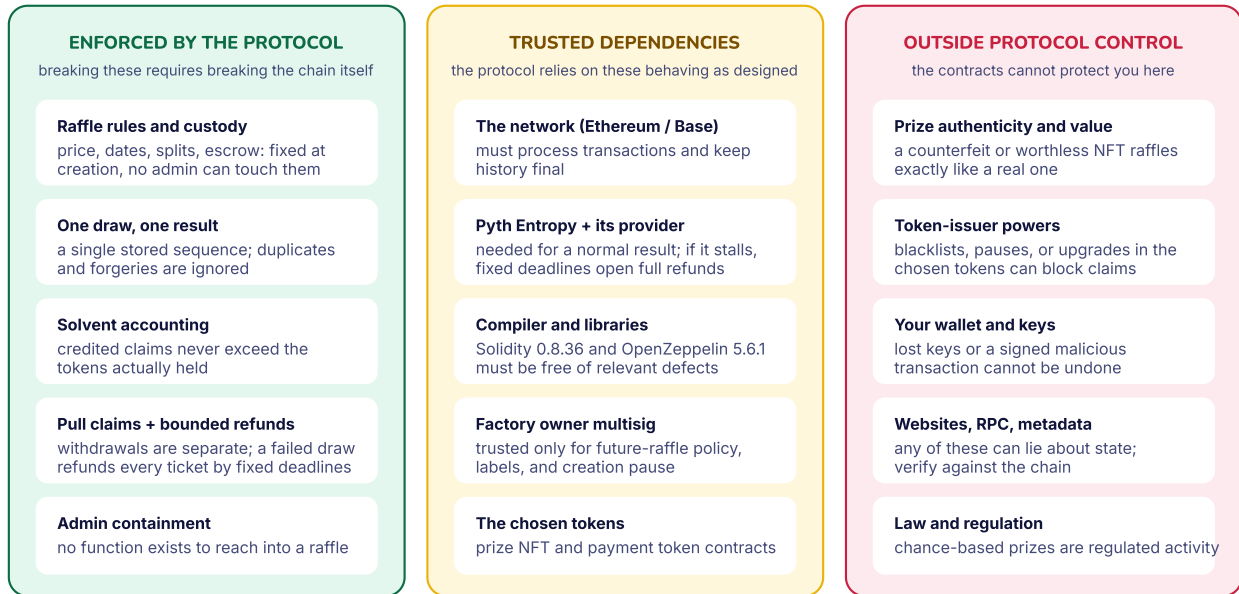
CHAPTER 31

The Trust and Dependency Map

Every system trusts something. Security is knowing exactly what, and what happens if that trust fails.

FIGURE 15 The trust and dependency map

Every guarantee in this document lives in the left column only. The middle column can delay or halt outcomes; the right column can cost you money.



Read each column left to right as decreasing protocol control. A raffle is only as trustworthy as its weakest relevant dependency.

Three rings of decreasing protocol control. The left ring is enforced by the code onchain; breaking it means breaking the chain or the code itself. The middle ring is real dependencies with designed failure modes. The right ring is beyond the protocol's reach entirely, and is where users must protect themselves.

For each dependency, what can go wrong, what the protocol does about it, and what remains yours to carry:

- **The underlying network (Ethereum or Base):** can reorganize recent blocks, censor, or halt. The protocol inherits whatever its chain does; deadlines assume the chain keeps producing blocks. Mitigation: none possible in-contract; the network's own security model applies.
- **Solidity and OpenZeppelin:** a compiler or library bug would undermine any contract. Mitigation: pinned, widely used versions (0.8.36, OZ 5.6.1) and a large shared blast radius that makes such bugs loudly public. Residual: not zero.
- **Pyth Entropy:** can delay, fail, or in the worst case be compromised. Delay and failure are designed for: fixed deadlines convert them into refunds. Compromised randomness (a provider able to bias results) is mitigated by Pyth's commit-reveal design and reputation, but is a genuine trust assumption for the fairness of the draw itself.
- **The chosen ERC-20:** issuer pause, blacklist, or upgrade can delay or block specific withdrawals. Mitigation: the admission gate keeps exotic tokens out, exact-transfer checks turn misbehavior into clean reverts, and claimants can direct claims to alternate addresses. Residual: issuer power is real power.

- **The chosen ERC-721:** can be counterfeit, mutable, or administratively controlled. Mitigation: escrow verification proves possession and honest transfer mechanics at creation. Residual: authenticity and continued behavior are not provable by the raffle.
- **Wallets, RPC, frontend, subgraph, metadata hosts:** all can lie to you or go dark; none can alter contract state. Mitigation: the authority hierarchy of Chapter 26, live re-reads, and simulation. Residual: a user who signs what a liar suggests.
- **The factory owner multisig:** bounded as Chapter 30 describes. Residual: future raffle policy and the admitted-token list.

CHAPTER 32

Threat Model

This chapter asks, for each participant in turn: what is the worst they could try, and what stops them? The table is a summary of the full internal threat model, which lives in the repository and drives the adversarial test suite.

HOSTILE ACTOR	REPRESENTATIVE ATTACK	WHAT STOPS IT
Sponsor	cancel after seeing weak sales	cancellation requires zero tickets sold, forever
Sponsor	raffle a counterfeit or self-controlled NFT	not stoppable in-contract; escrow verification proves possession only, and buyers must judge the collection (Chapter 36)
Sponsor	win their own raffle with bulk tickets	allowed and visible: purchases are public, odds are diluted openly, and the sponsor pays like anyone else
Buyer	pay less than the price via a taxed token	exact inbound balance-delta check reverts the purchase
Buyer	reenter the mint via a hostile receiver	reentrancy guard plus checks-effects-interactions
Buyer	buy after the close, or in the frozen states	exact state and timestamp checks
Ticket trader	snipe ownership between the draw request and the result	transfers are frozen for exactly that interval
Winner or claimant	claim twice, or redirect another's claim	claims zero their balance first; claim-for pays only the recorded account
Hostile claim recipient	revert on delivery to jam settlement	pull claims: only their own claim is affected, and it survives for retry
Payment token	skim, rebase, or misreport transfers	exact in/out delta checks; admission gate; clean reverts
Prize collection	lie about ownership at escrow	factory's post-transfer ownership verification fails creation
Callback forger	deliver a fake or duplicate random result	Entropy-only authentication, stored-sequence match, monotonic state

HOSTILE ACTOR	REPRESENTATIVE ATTACK	WHAT STOPS IT
Draw requester	grief by requesting with the minimum fee then nothing	the request is the useful action; the callback needs nothing further from them
Anyone	force value or tokens at the contract to distort accounting	explicit liability accounting; surplus is inert
Factory owner	any reach into a live raffle	no such function exists (Chapter 30)
Frontend or indexer	display false state to induce bad transactions	authority hierarchy, live re-reads, simulation; ultimately user vigilance
Transaction searchers	reorder purchases near the close, or race the timeout boundary	boundaries are exact and public; the race at the callback deadline is deliberately neutral between two valid outcomes

CHAPTER 33

What the Protocol Cannot Guarantee

This chapter exists so no reader can say the marketing buried the limits. None of these are edge cases the team plans to fix; they are boundaries of what any smart contract can promise.

- **Prize authenticity and value.** The contract escrows a token; whether it is the "real" artwork, whether its collection is legitimate, whether it holds any value next month, are all outside its knowledge.
- **Prize-collection behavior.** An upgradeable or admin-controlled NFT contract can freeze, alter, or reassign tokens while they sit in escrow. The raffle would still behave correctly; the prize inside it might not.
- **Payment-token issuer power.** Pauses and blacklists can delay or permanently block specific withdrawals. Refunds and claims are reserved and wait; the raffle cannot mint replacements.
- **Chain liveness and finality.** A halted or deeply reorganized network changes or suspends everything, deadlines included.
- **Oracle fairness.** The refund path bounds oracle unavailability, but the fairness of an actually delivered draw rests on Pyth's design holding.
- **Key custody.** Lost keys lose claims; there is no recovery desk. The recovery address exists precisely so sponsors can pre-commit a safer home for the prize.
- **User-side verification.** The protocol cannot stop a user from signing a transaction a counterfeit website suggested.
- **Metadata truthfulness.** Names, images, and descriptions travel outside the chain and can mislead; the contract sees none of it.
- **Legality.** Chance-based prize distribution is regulated activity in many places. Nothing here is a license, an opinion, or a defense.

- **Economic outcomes.** A legitimate raffle can still be a bad deal. Odds are transparent; value is your judgment.

CHAPTER 34

Testing and Review

What was actually done, reproduced at the reviewed commit, and what it does and does not prove.

The Foundry suite contains 69 tests: unit and boundary tests for every function and revert; a security suite driving adversarial mocks (reentrant tokens and receivers, false-returning and fee-skimming tokens, an outbound-tax token, a dishonest-transfer scenario, forced native value); 7 fuzz tests that hammer the arithmetic, the selection range, the threshold boundary, and the refund completeness with 1,000 random cases each locally and 10,000 in CI; and 9 stateful invariant tests that execute tens of thousands of random action sequences (256 runs of depth 64 locally, 1,000 of depth 256 in CI) while continuously asserting solvency, single-resolution, prize-escrow, refund-conservation, and monotonicity properties. The Hardhat suite adds 5 integration tests covering the Ignition deployment and two full journeys, one normal and one through grace expiry, refunds, and claim-for recovery, plus deployment-record validation. Static analysis (Slither, Solhint) is configured in CI, gas expectations are snapshot into the repository, and the oracle callback's measured gas is asserted to keep at least a 20 percent margin under its configured limit.

Coverage of the production contracts measures 96.88 percent of lines, 90.00 percent of branches, and 96.55 percent of functions, against a committed gate of at least 95 and 90. Mocks, scripts, and test code are excluded from that figure.

IMPORTANT RISK

None of this is an independent audit, and no quantity of testing proves the absence of vulnerabilities. The repository's own deployment runbook makes an external security review a hard gate before production. Until that happens, and until the contracts are deployed and observed under real conditions, treat every guarantee in this document as "designed and tested to," not "proven to."



PART VII

User Guides

Step-by-step walkthroughs for sponsors, buyers, and winners, and the checklists that keep them safe.

35 Sponsor Walkthrough

36 Ticket Buyer Walkthrough

37 Winner Walkthrough

38 Safety Checklists

Sponsor Walkthrough

You have an NFT and want to raffle it. Here is the whole journey, in order.

1. Verify you are in the right place. Confirm the application's domain carefully and, for meaningful amounts, cross-check the factory address the site uses against an independent source (the official announcement, the block explorer's verified-contract page). Everything else you do flows through that address.

2. Choose the prize and understand the approval. Pick the NFT. Your wallet will first ask you to approve the factory to transfer that specific token. Prefer single-token approval over collection-wide approval when the interface offers the choice.

3. Choose the payment token. The interface lists only tokens the factory currently admits. Pick the one your audience actually holds; a raffle priced in an obscure token is a raffle with fewer buyers.

4. Set the price and the minimum honestly. Price times minimum is the gross interest you are asking the market to show. Set the minimum at the level below which you would genuinely rather keep the NFT, because that is exactly what the contract will enforce. Remember the cash branch: if the minimum is missed, the winner takes 76 percent of whatever pot exists and you receive 19 percent plus the prize back.

5. Set the dates. Sales may start now or up to 7 days out, and run at most 30 days. Shorter windows concentrate attention; longer windows suit larger audiences. The 3-day draw window and 2-day callback timeout after closing are fixed and not yours to change.

6. Set the recovery address deliberately. This is where the prize returns in every ending except an outright win, and it cannot be changed later. A hardware or vault wallet you control is the robust choice; leaving it as your active wallet is the default.

7. Create, and check the result. One transaction does everything. Afterward, verify on the explorer that the new raffle owns your NFT and that its parameters read back as you intended. The interface will show the raffle's permanent address; share that address, not a screenshot.

8. During the sale. You have no dials to turn, which is the point and the pitch. You may buy tickets in your own raffle like anyone else, publicly. If nobody has bought yet and you regret the setup, you can cancel and reclaim the prize; the first sale removes that option permanently.

9. After the close. Anyone can request the draw; do not assume someone else will. If the raffle resolves, claim your proceeds with `claimQuote`; if the minimum was missed, your recovery address also reclaims the prize via `claimPrize`. If the draw fails entirely, expect refunds to be credited (anyone can do it, including you) and your recovery address to reclaim the prize. Claims never expire, but there is no reason to wait.

FOR SPONSORS

The one decision that deserves the most thought is the minimum. It is your public statement of reserve value, it decides which branch is likely, and it is the number sophisticated buyers will judge you by.

CHAPTER 36

Ticket Buyer Walkthrough

- 1. Judge the prize before the raffle.** The contract guarantees custody, not authenticity. Open the prize's collection on a block explorer or marketplace you trust: is it the canonical contract for that artwork or collectible, or a same-named copy? Does the collection have an owner or upgrade powers that could alter tokens? A minute of diligence here dominates everything else.
- 2. Judge the sponsor.** Pseudonymous is normal onchain; unverifiable is a warning. A sponsor whose address has history, whose socials link to the raffle address, and who answers questions is offering you more than a fresh wallet with a stock image.
- 3. Read the raffle's terms.** Fixed price, minimum, closing time, payment token: all onchain, all shown by the app, all checkable on the explorer. Note especially the minimum versus tickets already sold: it tells you which branch you are likely buying into, the NFT draw or the 76 percent cash draw.
- 4. Understand your odds, live.** Odds are tickets-held over tickets-sold, and tickets keep selling until the close. Ten percent odds now can be five percent tomorrow; the pot grows correspondingly. The app shows both continuously.
- 5. Approve and buy.** Your wallet approves the raffle to spend the exact purchase amount of the payment token, then the purchase mints your numbered tickets. Wallets with batching support show one confirmation; others show two. Verify the amount before signing; the contract will take exactly the advertised price and nothing else.
- 6. Optionally, move tickets.** Tickets are yours: transfer them to a friend, a cold wallet, wherever, any time before a draw request succeeds. Whoever holds a ticket when the draw lands, or when a refund is credited, receives what that ticket earns.
- 7. Wait for settlement.** After the close, someone (possibly you) requests the draw within 3 days; the result usually follows quickly. If instead the deadlines lapse, the raffle flips to refunds with no action needed from you except the final withdrawal.
- 8. Claim.** If you won, Chapter 37 is yours. If the raffle refunded, your balance appears once your tickets are credited; withdraw it with `claimQuote` whenever you like. Losing tickets in a resolved raffle earn nothing; they remain as souvenirs.

FOR TICKET HOLDERS

Spend only what you can afford to lose to chance. This is a raffle: the expected outcome of buying a ticket is losing the ticket price. The protocol makes the odds honest; it does not make them good.

CHAPTER 37

Winner Walkthrough

There are three ways to "win" and each ends differently.

You hold the winning ticket and the minimum was met. You are the recorded winner and prize claimant. Call `claimPrize` with the address where you want the NFT delivered (your everyday wallet, a vault, anywhere safe; not a contract that rejects NFTs, though if delivery fails the claim survives and you retry). A helper can alternatively push the prize straight to your recorded address using `claimPrizeFor`; nobody, helper or otherwise, can send it anywhere but to you.

You hold the winning ticket and the minimum was missed. You are the recorded winner of the cash prize: 80 percent of the post-fee pot. Call `claimQuote` with your chosen destination and the tokens transfer immediately. The NFT is not yours in this branch; it returns to the sponsor's recovery address.

The raffle failed and you held tickets. Not a win, but money back: once your tickets are credited (anyone can run the crediting; the app surfaces it), your full purchase price per held ticket is withdrawable via `claimQuote`, forever.

In every case: there is no deadline, no expiry, and no queue. Your claim is a row in the contract's storage that only your withdrawal (or a push to exactly you) can consume.

Safety Checklists

Sponsor checklist

- ☐ Domain and factory address verified against an independent source
- ☐ NFT approval granted for the single token, not the whole collection
- ☐ Payment token is one your audience actually holds
- ☐ Minimum set at your true reserve level; cash-branch split understood
- ☐ Sale window fits your audience; deadlines after close understood
- ☐ Recovery address is a wallet you control and will still control next month
- ☐ Raffle address verified on the explorer after creation; prize inside
- ☐ Plan for who requests the draw after closing

Buyer checklist

- ☐ Prize collection verified as canonical, not a same-named copy
- ☐ Collection checked for owner or upgrade powers over tokens
- ☐ Sponsor has history and answers questions
- ☐ Raffle address reached from a source you trust, not a random link
- ☐ Price, minimum, tickets sold, and closing time read and understood
- ☐ Which branch is likely (NFT or cash) and what the cash prize would be
- ☐ Approval amount matches the purchase exactly
- ☐ Only discretionary money spent; a ticket is a chance, not an investment



PART VIII

Frequently Asked Questions

Direct answers to the questions sponsors, buyers, and reviewers actually ask: twenty-six of them, each verifiable against the code.

Frequently Asked Questions

Answers below are statements about the contracts at the reviewed commit; each is verifiable in the code and cross-referenced in the fact-check record.

Who holds the NFT during the raffle?

The raffle contract itself. From the creation transaction until a terminal claim, the ledger lists the contract as the owner. No person, including the sponsor and the raffle.fun team, holds keys that can move it.

Can the sponsor take the NFT back after tickets are sold?

No. Cancellation requires zero tickets sold, permanently. After the first sale, the prize can only leave through the raffle's own endings.

Can the administrator choose or influence the winner?

No. Winner selection happens inside the randomness callback from the oracle's number, and no administrative function exists on a raffle at all.

Can anyone change the ticket price, minimum, or dates after creation?

No. There are no setter functions. The configuration written at creation is the configuration forever.

Is the minimum a cap? Can sales pass it?

The minimum is not a cap. Sales continue until the fixed closing time; each further ticket grows the pot and dilutes all odds. There is no maximum ticket count.

Can tickets be transferred, and who wins if a ticket changes hands?

Tickets transfer freely while the raffle is Active. Whoever holds the winning ticket at the instant the callback executes is the winner; for refunds, whoever holds a ticket when its refund is credited receives it. Transfers freeze during the draw and per-ticket during refunds precisely so these snapshots cannot be gamed.

What happens if not enough tickets sell?

The draw still happens. The winner receives 76 percent of the pot in cash, the sponsor 19 percent plus rounding, the treasury 5 percent, and the prize returns to the recovery address.

What happens if nobody buys a single ticket?

After the close, anyone may finalize the no-sales outcome and the recovery address reclaims the prize. No money existed, so nothing else needs to happen.

Who pays the randomness fee, and what if I overpay?

Whoever volunteers to request the draw pays the oracle's current fee in ETH. Any overpayment is credited back to that requester as a withdrawable balance.

What happens if the random number never arrives?

Deadlines take over. No successful request within 3 days of closing, or no callback within 2 days of a request, lets anyone flip the raffle into refunds: full ticket price back per ticket, no fee, prize home to the recovery address.

Are refunds automatic?

Almost. Entering refund mode and crediting each ticket are permissionless transactions someone must send (the app operator, a holder, anyone); your withdrawal is then yours to make whenever you wish. No step depends on a privileged party.

Why do I have to claim instead of just receiving my money?

So that nobody else's failure can block you. Push-payouts let one rejecting recipient jam an entire settlement; pull-claims isolate every participant. Chapter 23 is the full argument.

What if my claim transaction fails?

It reverts harmlessly and your claim survives. Retry any time, or claim to a different destination you control.

Can a malicious payment token trap funds?

A token that misdelivers makes transactions revert rather than corrupting balances, and only factory-admitted tokens can be used at creation. What remains is issuer power: a pause or blacklist can delay or block specific withdrawals, which no contract can cure. That residual risk is why the admission list exists.

Can any NFT be raffled?

Any token from a deployed contract that affirmatively reports ERC-721 support and honestly transfers ownership at escrow. Authenticity and value are explicitly not checked; judge the collection yourself.

Does the raffle.fun website control the raffle?

No. The website displays and assists. Every action it offers is a public contract function anyone can call with any wallet; if the site vanished, every raffle would proceed and settle unchanged.

Can the contracts be upgraded?

No. Raffles are non-upgradeable clones of a locked implementation. New protocol versions mean new contracts deployed separately; existing raffles never change.

What exactly does the factory owner control?

For future raffles only: the admitted payment-token list, the treasury address, and a creation pause. Nothing about any existing raffle. Chapter 30 is the complete inventory.

How are fees calculated, exactly?

Once, at resolution: 5 percent of the whole pot, rounded down, credited to the treasury. Failure and no-sales endings charge nothing. There are no other protocol fees; the draw requester separately pays the oracle's fee, and every transaction costs its sender ordinary gas.

Can the same ticket win twice, or a raffle draw twice?

No. One request, one accepted sequence, one winner, recorded once. Duplicates and replays are ignored by construction.

Is the last ticket sold actually eligible?

Yes. The formula (random mod tickets) plus 1 covers every ticket from 1 through the last, inclusive, each with equal probability; a one-ticket raffle picks ticket 1.

Can a sponsor rig the draw by picking the moment it happens?

Requesting earlier or later does not help: the random value comes from the oracle's commit-reveal process, not from block data, and the request locks transfers before any result exists. A sponsor can buy tickets like anyone else, publicly; that is participation, not rigging.

Is raffle.fun legal in my country?

This document cannot answer that. Chance-based prize distribution is regulated differently everywhere, and nothing here is legal advice. Obtain advice for your jurisdiction before operating or participating.

Has the protocol been independently audited?

No. Extensive internal testing exists (Chapter 34), and the deployment runbook requires an independent review before production, but at the reviewed commit no external audit has been performed.

What should I verify before interacting, in one breath?

The domain, the factory address, the raffle address, the prize collection's authenticity, the payment token, the price, the minimum versus tickets sold, and the closing time. The checklists in Chapter 38 expand each item.



PART IX

Conclusion

What this design actually makes possible, and the handful of facts worth remembering.

39 What raffle.fun Makes Possible

40 Final Takeaways

What raffle.fun Makes Possible

Strip away the terminology, and raffle.fun does one specific thing: it lets a stranger run a prize drawing that other strangers can join without anyone needing to believe anyone. The prize is provably locked before the first ticket sells. The rules are provably frozen before the first ticket sells. The odds are publicly countable at every moment. The draw comes from a source none of the participants control, exactly once. The payouts follow arithmetic that was published in advance, and every failure the designers could bound is bounded: a raffle that cannot finish returns everyone's money by rules, not by customer support.

That is a narrow claim, and the narrowness is deliberate. The protocol does not judge prizes, guarantee value, replace law, or eliminate every dependency; Chapters 31 and 33 held that line honestly. What it eliminates is the specific, historically well-earned need to trust a raffle operator with custody, rules, odds, drawing, and settlement all at once. For sponsors, a credible raffle no longer requires a reputation as collateral. For participants, "will I get paid?" is answered by code you can read instead of a promise you cannot. For builders, raffle mechanics become infrastructure: a primitive to compose with, audit once, and reuse.

Whether that primitive becomes widely used depends on things no whitepaper can settle: audits, deployment, real communities, and law. What this document establishes is what the software does, precisely, at one commit, so every later conversation can start from facts.

Final Takeaways

Seven sentences to keep:

1. The raffle contract, not any person, holds the prize and the money from creation to settlement, and its rules cannot be edited by anyone after creation.
2. Every ticket has equal odds, the count is public, and the minimum is an outcome threshold, never a sales cap.
3. One random result from Pyth Entropy decides the winner; equality with the minimum counts as met, the last ticket is eligible, and no draw can be redone.
4. Money flows are fixed: a 5 percent fee, then 100/0 or 80/20 of the rest by threshold; failed draws refund every ticket in full and charge nothing.
5. Everything is pull-based and permissionless where it matters: anyone can request the draw, finalize a failure, or help deliver a claim, and claims never expire.

6. The administrator's power stops at the door of every live raffle, structurally: the functions to interfere do not exist.
7. The honest limits remain: unaudited code, prize authenticity, token-issuer power, oracle fairness, your own keys, and your own jurisdiction's law.



APPENDICES

Technical Reference

The exact state machine, formulas, contract surfaces, events, invariants, asset requirements, glossary, and sources.

A Exact State Machine

B Economic Formulas

C Contract Reference

D Event Guide

E Security Invariants

F Supported Asset Model

G Glossary

H References

Exact State Machine

States: `Uninitialized`, `AwaitingPrize`, `Active`, `DrawRequested`, `Resolved` (terminal), `Cancelled` (terminal), `Refunding` (terminal). Terminal means the outcome can never change; claim and refund-credit activity continues inside terminal states. Figure 6 in Chapter 12 draws this table.

#	FROM	TO	FUNCTION	CALLER	CONDITIONS	SIDE EFFECTS	EVENT
1	Uninitialized	AwaitingPrize	<code>initialize</code>	factory only	caller is a contract whose <code>raffleImplementation()</code> equals the clone's embedded implementation; all parties and dependencies nonzero	full configuration stored; ERC-721 ticket collection initialized	none (creation is evented by the factory)
2	AwaitingPrize	Active	<code>onERC721Received</code>	prize contract	operator is the factory, <code>from</code> is the sponsor, exact prize contract and token ID	none beyond the state	PrizeDeposited
3	Active	Cancelled	<code>cancelBeforeSales</code>	sponsor only	<code>totalTickets == 0</code> (no time condition)	outcome <code>CancelledBeforeSale</code> ; prize claimant := recovery address	RaffleCancelled
4	Active	Resolved	<code>closeNoSales</code>	anyone	<code>now >= endTime</code> and <code>totalTickets == 0</code>	outcome <code>NoSales</code> ; prize claimant := recovery address	NoSalesClosed
5	Active	DrawRequested	<code>requestDraw</code>	anyone	<code>now >= endTime</code> , <code>now < endTime + 3 days</code> , <code>totalTickets > 0</code> , <code>msg.value >= current Entropy fee</code>	<code>drawRequestedAt := now</code> ; exact fee forwarded to Entropy; sequence stored; excess credited to requester as native claim	DrawRequested
6	Active	Refunding	<code>finalizeUnrequestedDraw</code>	anyone	<code>totalTickets > 0</code> and <code>now >= endTime + 3 days</code>	outcome <code>DrawNotRequested</code> ; pot moved to refund liability; prize claimant := recovery address	DrawFailureFinalized
7	DrawRequested	Resolved	<code>entropyCallback</code>	Entropy contract only (via Pyth wrapper)	sequence equals stored sequence; no in-flight request	winning ticket and owner snapshotted; fee, winner, and sponsor credits recorded per branch; pot zeroed; outcome <code>NftAwarded</code> or <code>CashFallback</code> ; prize claimant := winner or recovery address	RaffleResolved

#	FROM	TO	FUNCTION	CALLER	CONDITIONS	SIDE EFFECTS	EVENT
8	DrawRequested	Refunding	finalizeTicket drawTimeoutDraw	anyone	now >= drawRequestedAt + 2 days	outcome DrawTimedOut ; same effects as 6	DrawFailureFinalized

Rejected callback deliveries (wrong sequence, wrong state, in-flight) emit `EntropyCallbackIgnored` and change nothing. At the timeout boundary (row 7 versus row 8), both transactions are valid until one executes; the first included wins and the other becomes a no-op or ignored event.

Non-transition actions, by state:

ACTION	ALLOWED STATES	CALLER	CONDITIONS AND EFFECTS
<code>buyTickets</code>	Active	anyone	<code>startTime <= now < endTime</code> ; quantity 1..100; exact gross payment verified; contiguous ticket IDs minted to the recipient
ticket transfer	Active, Resolved, Cancelled; Refunding only for credited tickets	ticket owner or approved	owner-to-owner moves revert in <code>DrawRequested</code> (<code>TicketTransfersFrozen</code>) and for uncredited refund tickets (<code>RefundTicketFrozen</code>)
<code>creditTicketRefunds</code>	Refunding	anyone	1..100 valid, not-yet-credited ticket IDs; credits exactly one <code>ticketPrice</code> per ticket to its current owner; no external calls
<code>claimQuote</code> / <code>claimQuoteFor</code>	any state (balances exist only after resolution or crediting)	claimant / anyone	zeroes the balance, then transfers with exact debit and credit verification; <code>-For</code> pays only the account itself
<code>claimPrize</code> / <code>claimPrizeFor</code>	Resolved, Cancelled, Refunding	prize claimant / anyone	single-use flag set before transfer; failed delivery reverts atomically; <code>-For</code> delivers only to the recorded claimant
<code>claimNative</code>	any	accrued account	pays draw-fee overpayment credits to a chosen destination

APPENDIX B

Economic Formulas

All arithmetic is integer arithmetic on raw token units. `floor` division is Solidity's default; multiplication happens before division via `Math.mulDiv`, so no intermediate overflow or premature truncation occurs. Constants: `BPS = 10,000`, `PROTOCOL_FEE_BPS = 500`, `CASH_WINNER_BPS = 8,000`.

purchase:	gross	= <code>ticketPrice x quantity</code>
resolution:	fee	= <code>floor(pot x 500 / 10,000)</code>
	distributable	= <code>pot - fee</code>
threshold met:	sponsorCash	= <code>distributable ; winnerCash = 0</code>
threshold missed:	winnerCash	= <code>floor(distributable x 8,000 / 10,000)</code>
	sponsorCash	= <code>distributable - winnerCash</code>
refunding:	refundLiability	= <code>pot ; per ticket: ticketPrice ; fee = 0</code>

Accounting identities, maintained continuously and checked by invariant tests:

```
accountedQuote = unsettledPot + uncreditedRefundLiability + totalClaimableQuote
solvency       : quoteToken.balanceOf(raffle) >= accountedQuote
conservation    : fee + winnerCash + sponsorCash = pot          (resolved)
                 sum over tickets of ticketPrice = pot          (refunding)
native         : address(raffle).balance >= totalClaimableNative
```

Rounding behavior: `fee` and `winnerCash` round down; `sponsorCash` is a subtraction and therefore absorbs all rounding remainders (at most 2 raw units across both floors). The fee is computed once on the aggregate pot, so purchase splitting cannot alter total fees by even one unit.

Verified vectors (six-decimal token; raw units shown for the dust case):

VECTOR	POT	FEE	DISTRIBUTABLE	WINNER CASH	SPONSOR CASH
120 × 10.00, met	1,200.000000	60.000000	1,140.000000	0 (NFT)	1,140.000000
100 × 10.00, met exactly	1,000.000000	50.000000	950.000000	0 (NFT)	950.000000
99 × 10.00, missed	990.000000	49.500000	940.500000	752.400000	188.100000
80 × 10.00, missed	800.000000	40.000000	760.000000	608.000000	152.000000
1 × 10.00, missed	10.000000	0.500000	9.500000	7.600000	1.900000
10 × 0.333333, missed (raw)	3,333,330	166,666	3,166,664	2,533,331	633,333

Each vector reproduces with `calculateResolutionAmounts` in `packages/sdk/src/math/economics.ts` and matches the contract worked-example tests.

APPENDIX C

Contract Reference

Raffle (the per-raffle clone)

Purpose: escrow one prize, sell tickets, settle one outcome, hold claims. Trust model: no owner, no admin, no upgrade; only the factory (at initialization) and the configured Entropy contract (at callback) are privileged, each for one call.

Key storage: configuration (factory, sponsor, recovery address, treasury, quote token, entropy, prize, price, minimum, times, callback gas, metadata URI); counters (`totalTickets` , `grossSales` , `unsettledPot` , `uncreditedRefundLiability` , `totalClaimableQuote` , `totalClaimableNative`); settlement (`entropySequenceNumber` , `drawRequestedAt` , `winningTicketId` , `winner` , `prizeClaimant` , `state` , `outcome` , `prizeClaimed`); per-account claim mappings and the per-ticket refund-credited mapping.

External surface (state-changing): `initialize`, `buyTickets`, `cancelBeforeSales`, `closeNoSales`, `requestDraw`, `finalizeUnrequestedDraw`, `finalizeTimedOutDraw`, `creditTicketRefunds`, `claimQuote`, `claimQuoteFor`, `claimPrize`, `claimPrizeFor`, `claimNative`, plus ERC-721 transfers of tickets. Views include state, deadlines (`requestGraceDeadline`, `callbackDeadline`), `getEntropyFee`, odds, solvency and surplus inspectors, and refund-credit status.

External calls made: quote-token transfers (guarded by exact-delta checks), the prize collection's `safeTransferFrom` (at claim), Entropy's `getFeeV2` and `requestV2`, and ticket recipients' ERC-721 receiver hooks (bounded by quantity and the reentrancy guard). Native transfers occur only in `claimNative` and `requestDraw`'s fee forwarding. The `receive` function reverts.

Errors are custom and specific (over two dozen, from `SaleNotStarted` to `RefundTicketFrozen`), which keeps reverts cheap and diagnosable; Appendix D lists the events.

RaffleFactory

Purpose: create, register, and configure raffles; administer future creation. Trust model: `Ownable2Step` owner (multisig expected) with the three powers of Chapter 30; holds no assets; `nonReentrant` creation.

Storage: immutable implementation, Entropy, callback gas limit; current treasury, raffle count, creation pause; registries (`raffleById`, `idByRaffle`, `isRaffle`); the bounded admitted-token list with stable indices. Creation validates inputs, deploys the deterministic clone, initializes it, registers it, emits `RaffleCreated`, escrows the prize, and verifies real ownership, all atomically.

RaffleLens

Purpose: one-call aggregated reads for registered raffles (single and batches up to 100), including deadlines, liabilities, per-account claims and ticket balances, action availability flags, and an `entropyFeeAvailable` indicator so a broken oracle fee read cannot hide recovery actions from interfaces. Trust model: stateless, immutable factory binding, rejects unregistered addresses, cannot write.

APPENDIX D

Event Guide

A raffle's complete history reconstructs from events alone, which is exactly what the subgraph does. In lifecycle order:

EVENT (EMITTER)	FIRED WHEN	KEY FIELDS
<code>QuoteTokenVerificationUpdated</code> (factory)	owner admits or removes a payment token	token, previous and new status

EVENT (EMITTER)	FIRED WHEN	KEY FIELDS
ProtocolTreasuryUpdated , CreationPauseUpdated , ownership events (factory)	admin policy changes	previous and new values
RaffleCreated (factory)	a raffle is created	raffle ID and address, sponsor, recovery address, prize, quote token, treasury, price, minimum, normalized start, end, request deadline, metadata
PrizeDeposited (raffle)	the exact prize enters escrow	prize contract, token ID, sponsor
TicketsPurchased (raffle)	each purchase	buyer, recipient, quantity, first and last ticket ID, gross amount
Transfer (raffle, ERC-721)	each ticket mint or move	standard ERC-721 fields; mints come from the zero address
RaffleCancelled / NoSalesClosed (raffle)	the two zero-sales endings	sponsor or caller, prize claimant
DrawRequested (raffle)	the one accepted request	sequence, requester, fee, excess credited, request time, callback deadline
EntropyCallbackIgnored (raffle)	a rejected delivery	received and expected sequence, current state
RaffleResolved (raffle)	the callback settles	sequence, winning ticket, winner, outcome, prize claimant, fee, winner cash, sponsor cash
DrawFailureFinalized (raffle)	a failure deadline is exercised	outcome, caller, recovery address, gross refund liability
TicketRefundCredited (raffle)	each ticket's refund credit	ticket ID, credited owner, amount, remaining liability
QuoteClaimed / PrizeClaimed / NativeClaimed (raffle)	each withdrawal	account, destination, amount or token ID

Reconstruction rules an indexer follows: creation precedes the deposit in the same transaction (the factory registers and emits before escrow); purchase events pair with mint Transfers; exactly one of `RaffleResolved`, `DrawFailureFinalized`, `NoSalesClosed`, or `RaffleCancelled` is terminal per raffle; refund credits sum to the recorded liability; and claim events monotonically consume recorded credits. Every event ID derives from transaction hash plus log index, so duplicate delivery is idempotent.

APPENDIX E

Security Invariants

Each property is stated formally, then in everyday terms, and each is continuously attacked by the stateful invariant suite (256 random call sequences of depth 64 locally; 1,000 of depth 256 in CI).

FORMAL INVARIANT (TEST NAME)	IN EVERYDAY TERMS
<code>invariantAccountedQuoteAlwaysReconcilesAndIsSolvent</code>	The contract's token balance always covers the pot plus uncredited refunds plus all recorded claims. The books never show money that is not there.
<code>invariantQuotePaidInEqualsPaidOutPlusContractBalance</code>	Every unit that ever entered equals what left through claims plus what remains. Nothing leaks and nothing is minted.
<code>invariantPrizeLeavesEscrowAtMostOnceAndOnlyAfterAClaimPathExists</code>	The prize moves at most once, and only after a terminal outcome named its claimant.
<code>invariantAtMostOneRequestAndResolutionExist</code>	One randomness request, one resolution, ever.
<code>invariantResolvedWinnerIsAlwaysARealSoldTicket</code>	A recorded winner always corresponds to a ticket that was actually sold.
<code>invariantEverySoldTicketHasANonzeroOwner</code>	No ticket is ever ownerless; the selection formula always lands on a real holder.
<code>invariantResolutionBranchMatchesExactThresholdBoundary</code>	The branch taken always matches <code>tickets >= minimum</code> , including exact equality.
<code>invariantRefundingConservesGrossAndNeverCreditsProtocolFee</code>	In refund mode, credited refunds plus remaining liability always equal the original pot, and no fee is ever taken.
<code>invariantStateTransitionsNeverMoveBackward</code>	The state machine is monotonic: no sequence of calls re-opens, re-rolls, or rewinds a raffle.

Two further properties are enforced by construction and unit tests rather than a dedicated invariant: donations and forced value are inert surplus, and no administrator path into a live raffle exists (no such function is present in the bytecode).

APPENDIX F

Supported Asset Model

The protocol's guarantees hold for assets within this envelope. Outside it, the design degrades safely (transactions revert) but user experience may not.

Payment tokens (ERC-20). Supported: contract-backed tokens that transfer exact amounts in both directions with no fee-on-transfer, no rebasing or elastic supply, no transfer hooks that reenter, and standard `balanceOf` / `transfer` / `transferFrom` semantics (return-value quirks are tolerated via SafeERC20). The factory admission list is the operational enforcement of this envelope; the exact-delta checks are the mechanical enforcement. Explicitly unsupported: taxed or reflective tokens, rebasing tokens, tokens whose balances change without transfers. Issuer controls (pause, blacklist, upgrade) are compatible with correctness but can delay or block individual withdrawals.

Prize collections (ERC-721). Supported: deployed contracts that affirmatively report ERC-721 support via ERC-165, implement honest ownership (`ownerOf` reflects transfers), and perform standard `safeTransferFrom`. The escrow handshake verifies these properties at creation. Explicitly

unsupported: collections that lie about ownership (creation reverts), and semi-fungible or fractional standards such as ERC-1155. Mutable, pausable, or admin-controlled collections pass the handshake if they behave at that moment; their later behavior is outside the guarantee, as Chapter 33 states.

Native currency. Used only to pay the Entropy fee through `requestDraw`. Direct sends revert; force-pushed value is inert surplus.

APPENDIX G

Glossary

TERM	MEANING
Base	An Ethereum layer-2 network with low fees; a raffle.fun target chain alongside Ethereum itself.
Ethereum	The largest smart-contract blockchain and a raffle.fun target network.
Blockchain	A public ledger maintained by many computers that no single party can rewrite.
Callback	The second half of the randomness exchange: Entropy's transaction delivering the result.
Claim (pull payment)	A recorded debt the claimant withdraws themselves; the contract never pushes assets unprompted.
Clone (EIP-1167)	A minimal contract that borrows logic from a shared implementation while keeping its own storage.
Commit-reveal	A scheme where parties commit to hidden values first, so no one can choose their value after seeing others'.
CREATE2	A deployment method making contract addresses predictable in advance.
Distributable pot	The pot after the 5 percent fee: what winner and sponsor share.
ERC-20	The standard for interchangeable tokens (money-like balances).
ERC-165	The standard by which a contract declares which interfaces it supports.
ERC-721	The standard for non-fungible (unique) tokens; both prizes and tickets follow it.
Escrow	Custody by a neutral keeper until rules decide the owner; here, the raffle contract itself.
Entropy (Pyth)	The external randomness service delivering the draw's random number.
Factory	The contract that creates, registers, and configures raffles.
Gas	The network fee paid for executing a transaction.
Grace period	The 3-day window after closing during which the draw may be requested.
Immutable	Unchangeable after deployment; raffle configuration and code are immutable.
Indexer / subgraph	Offchain software that turns contract events into a searchable database.
Lens	The read-only contract aggregating raffle state for applications.
Metadata	Offchain descriptive data (names, images) referenced by tokens; not authoritative.

TERM	MEANING
Minimum (threshold)	The ticket count at or above which the winner receives the NFT rather than cash.
Multisig	An account requiring multiple signatures; the intended factory owner.
NFT	Non-fungible token: a unique blockchain asset under ERC-721.
Oracle	A service that brings offchain information (here, randomness) onchain.
Permissionless	Callable by anyone; no allowlist or role required.
Pot (unsettled pot)	All ticket payments held by the raffle awaiting settlement.
Quote token	The raffle's one payment token, in which the price is quoted.
Recovery address	The fixed destination that reclaims the prize in every non-winner ending.
Reentrancy	An attack where an external call re-enters the caller mid-operation; guarded throughout.
Refunding	The terminal state after a failed draw in which every ticket is credited its price back.
RPC provider	The service relaying your reads and transactions to the network.
Sequence number	The unique ID of the raffle's one randomness request.
Smart contract	A program on the blockchain that runs exactly as written.
Sponsor	The raffle's creator and prize depositor.
Terminal state	A state from which the outcome can never change (Resolved, Cancelled, Refunding).
Transaction	A signed instruction executed by the network.
Treasury	The fixed address credited the protocol fee.
Wallet	Software holding your signing keys and submitting transactions.

APPENDIX H

References

Primary sources only; all specification claims in this document trace to one of these or to the repository itself.

- ERC-20 Token Standard: eips.ethereum.org/EIPS/eip-20
- ERC-165 Standard Interface Detection: eips.ethereum.org/EIPS/eip-165
- ERC-721 Non-Fungible Token Standard: eips.ethereum.org/EIPS/eip-721
- EIP-1167 Minimal Proxy Contract: eips.ethereum.org/EIPS/eip-1167
- Solidity documentation (v0.8.36): docs.soliditylang.org
- OpenZeppelin Contracts 5.x documentation: docs.openzeppelin.com/contracts/5.x
- Pyth Entropy documentation: docs.pyth.network/entropy
- Ethereum documentation: ethereum.org/en/developers/docs

- Base network documentation: docs.base.org
 - The raffle.fun repository at reviewed commit `a2120f5e163dc3641d9864773febbfedca047edb` : contracts under `packages/contracts/src` , tests under `packages/contracts/test` , protocol documentation under `docs/` , and the claim-by-claim record in `docs/whitepaper/FACT-CHECK.md` .
-

This document is complete as of August 9, 2026, describing protocol version 1.0.0 at the reviewed commit. Corrections and questions: open an issue in the repository or contact the maintainers through the security policy for sensitive reports.